UNIVERSIDAD NACIONAL DEL CENTRO DE LA PROVINCIA DE BUENOS AIRES
FACULTAD DE CIENCIAS EXACTAS
TRABAJO FINAL DE LA CARRERA INGENIERÍA DE SISTEMAS



Plataforma de Experimentación A/B en Redes de Entrega de Contenidos

Guillermo Pereyra Irujo

Directora: Dra. Mariana del Fresno
Co-director: Ing. Pablo Daniel Del Grosso

Tandil, Argentina, febrero de 2015

Resumen

Para mantener la competitividad en términos de performance, un sitio web con público en todo el mundo necesita una red global de entrega que distribuya su contenido. Esta red atiende los pedidos disminuyendo la distancia media hasta el cliente, pero también quita la posibilidad a los servidores de aplicación de procesar la información de cada pedido. La técnica de experimentación A/B requiere el ajuste del contenido en cada respuesta. Se puede implementar del lado del cliente, pero hay limitaciones técnicas que degradan la calidad del producto y afectan el objeto de análisis. Se puede implementar también del lado del servidor, pero sólo si éste recibe todos los pedidos. Cuando se usa una red de entrega de contenidos, entonces, no hay una buena opción de implementación de la técnica de experimentación A/B. En este trabajo se analizan tales opciones y los inconvenientes de su implementación, y se propone una implementación alternativa.

La propuesta consiste en utilizar nuevas capacidades de las redes de entrega de contenido para incluir procesamiento de datos de entrada en el proceso de distribución de la red misma, descargando a la vez al servidor de procesar todos los pedidos y al cliente de disminuir su performance afectando el producto. Se describen las alternativas evaluadas para cada componente de la solución y los pasos del proceso iterativo de desarrollo que permitió alcanzarla. La solución consiste en una herramienta de variación de contenido que permite evaluar las hipótesis de trabajo, sumada a un componente de recolección de datos que muestrea las exposiciones y acciones de los usuarios y un componente de reporte que sumariza los resultados orientando la presentación a la asistencia en toma de decisiones. Como valor agregado, se incluye la automatización de la distribución de las asignaciones, de manera que la correcta medición de correlaciones y resultados marginales no requiera del operador conocimientos especializados.

Índice

Secciones

Resumen 2

1. Introducción 7

1.1. Motivación 8

1.2. Organización del trabajo 9

2. Definición del problema 10

2.1. La técnica de experimentación A/B 10

2.2. Herramientas de experimentación A/B 11

2.3. Variación de contenido en el servidor 12

2.4. Variación de contenido en el cliente 17

2.5. Variación de contenido en la CDN 24

3. Desarrollo de la propuesta 26

3.1. Componentes 26

3.2. Alternativas evaluadas 31

3.2.1. Selección de páginas completas o variación de fragmentos 31

3.2.2. Evaluación en una pasada o en dos pasadas 35

3.2.3. Tracking del dado o de la variación 48

3.3. Software desarrollado 59

4. Resultados 62

5. Conclusión y trabajo futuro 65

Bibliografía 67

Tablas y figuras

Tabla 1. Implementaciones de experimentación A/B disponibles en el mercado. 11

Tabla 2. Ejemplos de circulación entre páginas, con conversiones y variaciones. 51

Figura 1. Esquema de una red global de conmutación de paquetes con clientes, servidores y cachés. Las cachés sustituyen a los servidores respondiendo con copias de respuestas anteriores mientras aún son válidas. 13

Figura 2. Esquema de una red global de conmutación de paquetes con clientes, servidores y cachés. Puesto que la frecuencia de comunicación entre clientes y cachés es igual o mayor que aquella entre cachés y servidores, ubicar las cachés cerca de los clientes disminuye el tiempo promedio de respuesta. 14

Figura 3. Pilas de protocolos de servidor, caché y cliente en una comunicación HTTP intermediada por una CDN. 15

Figura 4. Variación de contenido en el cliente. El contenido original del documento es la versión de control y un script sujeto a la condición de tratamiento cambiar el contenido tan pronto el documento es modificable. 17

Figura 5. Retraso de la aplicación del tratamiento. El contenido original se muestra inmediatamente, pero el tratamiento se aplica sólo después de la carga del recurso externo, cuya demora es usualmente mayor que el período de rendering. 19

Figura 6. Espera activa hasta la aparición del objetivo. Tan pronto hay una coincidencia con la búsqueda, se aplica la variación y finaliza la espera. Si, más tarde, llegan nuevos elementos que coinciden con la búsqueda, no resultan variados. 20

Figura 7. Espera activa hasta que el documento esté completo, pero con una transformación no idempotente. En este caso, cada elemento EM es envuelto entre comillas tantas veces como iteraciones ocurran, en lugar de sólo una vez. 22

Figura 8. Ejemplo de documento ESI, que produce un documento HTML dinámico, dependiente de la presencia de una cookie. 27

Figura 9. Tirada y almacenamiento del dado y selección de la variación de un experimento, implementadas en ESI. 28

Figura 10. Selección de contenido a emitir a la salida en función de la variación seleccionada para un experimento. 29

Figura 11. Primera implementación. La respuesta en la URL original es el documento ESI que incluye los dos contenidos alternativos pero selecciona sólo uno para la salida. La respuesta en las URLs con el parámetro adicional son las variaciones. 31

Figura 12. Segunda implementación. La respuesta en la URL original es un 33

documento ESI con el contenido original no variado y las variaciones envueltas en condicionales.

Figura 13. Para emitir todas las variaciones en un mismo documento, hay que ejecutar en el servidor todas las ramas de la selección. La interfaz programática es entonces una función que recibe dos bloques de código, uno por tratamiento, uno por control. 34

Figura 14. Pseudocódigo de la función de variación, que recibe la versión a la que debe condicionar las ramas, y una función de retrollamada para cada rama. 34

Figura 15. Primera implementación. Lectura y almacenamiento del dado, selección de las variaciones, y condicionamiento de la emisión del contenido. 35

Figura 16. Pseudocódigo de dos páginas, una (página principal) en la que se varía por un experimento (testAvisoAnuncio), otra (página de publicación) en la que se varía por otro experimento (testBotónLink). La selección de variaciones se evalúa para todos los experimentos activos, independientemente de si hay condicionamientos a ellos en la respuesta. 36

Figura 17. Pseudocódigo de dos páginas, una (página principal) en la que se varía por un experimento (testAvisoAnuncio), otra (página de publicación) en la que se varía por otro experimento (testBotónLink). La selección de cada variación ocurre por demanda inmediatamente antes del primer condicionamiento. 37

Figura 18. Pseudocódigo de una página con dos bloques condicionados, con la selección de variación inmediatamente antes del primer bloque, y una condición externa afectando todo el primer bloque. Si la condición externa no se cumple, en el segundo bloque no habrá información suficiente para decidir. 39

Figura 19. Definición declarativa de dos experimentos, incluyendo la lista de páginas que cada uno afecta, y pseudocódigo de tales páginas, cada uno con la evaluación de sólo aquellos experimentos a los cuales tiene código condicionado, nuevamente al inicio. 40

Figura 20. Pseudocódigo de una página con una variación condicionada a un valor de entrada. Es imposible saber a priori si la variación estará presente en la salida, para emitir o no, antes, la selección de la variación. 42

Figura 21. Pseudocódigo de una página, con una variación de contenido visible, luego la exportación a javascript de la lista de experimentos activos, después la rutina de tracking, y finalmente el resto de los scripts. 45

Figura 22. Pseudocódigo solución. Una función principal con todo el contenido que debe ser emitido al usuario, menos los bloques de evaluación y exportación, expresados como llamados a funciones de retrollamada que inyectan el código restante. De esta manera, pueden ser emitidos a la salida del servidor después del contenido, pero emitidos a la salida de la CDN antes. 47

Figura 23. Pseudocódigo de la implementación final de la solución, sujeta a las limitaciones del lenguaje. 48

Figura 24. Ámbitos de etiquetado de registros en Google Analytics. 50

Figura 25. Autómata finito que representa un conjunto de páginas que forman un sitio básico de avisos clasificados y la posible circulación entre ellas. 51

Figura 26. Un usuario participa en un experimento X y le queda el dado tirado. Luego convierte una meta del experimento Y sin pasar por la exposición. La conversión es erróneamente atribuida a la variación de Y. 52

Figura 27. La misma situación que en la figura 26, pero al no estar la marca de exposición en el registro de la conversión, la conversión no resulta atribuida al experimento. 53

Figura 28. Un usuario es expuesto a una variación del experimento X y su conversión en la siguiente página es correctamente atribuida. Luego es expuesto a una variación del experimento Y por la cual ocurre otra conversión, pero la exposición intermedia a otro experimento sobre escribe la marca. 53

Figura 29. La misma situación que en la figura 28 pero, almacenando toda la lista de exposiciones en la marca, ambas conversiones son correctamente atribuidas. 54

Figura 30. Distribución del espacio de valores del dado entre las variaciones de diferentes experimentos para eludir la correlación. 55

Figura 31. Al tirar un dado por experimento, fue necesario almacenar, además de la ocurrencia de la exposición, la identificación de la variación a la cual el usuario había sido expuesto. El resultado es una lista de pares experimento:variación. 56

Figura 32. Almacenamiento de un conjunto de 64 dados de dos dígitos como un string de 128 caracteres con números del 0 al 9. 56

Figura 33. Evaluación de un experimento al que se ha asignado el dado 35. 57

Figura 34. Evaluación de un experimento al que se ha asignado el dado 35 y el desplazamiento 68. 57

Figura 35. Una función de dispersión distribuye uniformemente los experimentos entre los dados y en el espacio de valores de cada dado. Una vez determinados el dado y el desplazamiento, el valor resultante indica la variación para el usuario. 58

1. Introducción

En el mundo del marketing, frecuentemente se recurre a la técnica de experimentación A/B para optimizar un producto o servicio con pasos pequeños y certeros. Esta técnica consiste en hacer una pequeña variación en el producto o servicio y exponerla a una porción del público, midiendo las variables de interés del negocio, como por ejemplo la cantidad de ventas o la eficiencia en el uso de los recursos. Cuando los valores resultantes son significativamente mejores en el producto o servicio variado, se considera que la variación es positiva para el negocio y se incorpora definitivamente al producto o servicio. Cuando los valores son peores o la diferencia no es significativa, la variación se descarta. En el mundo de la World Wide Web (o WWW), la técnica se implementa modificando una o más páginas de un sitio y repartiendo a los usuarios entre la versión original y la versión modificada. Las variables a medir en una y otra versión serán las reacciones de los usuarios, como cantidad de visitas, tiempo de permanencia en el sitio o clics en elementos del contenido.

La WWW funciona sobre una arquitectura cliente-servidor. En cada interacción, el servidor tiene almacenado, o genera por demanda, el contenido de una página web y lo transfiere al cliente al otro lado de la red. La modificación del contenido para ejecutar un experimento A/B se puede hacer del lado del cliente y del lado del servidor. Si se hace del lado del cliente, el servidor responderá siempre el mismo contenido y cada cliente, al recibirlo, hará los cambios necesarios para que el usuario vea la versión que le corresponde. Si se hace del lado del servidor, en cada pedido se generará la versión de la página web que le corresponda al cliente que la pide, y el contenido será transferido listo para ver.

Sin embargo, la tecnología necesaria para abstraer completamente el proceso de modificación del lado del cliente no está aún disponible: los guiones (o *scripts*) que permiten ejecutar acciones del lado del cliente no tienen primitivas de sincronización con las rutinas de presentación (o *rendering*), por lo que se dan condiciones de carrera que afectan la experiencia del usuario y, por lo tanto, los resultados del experimento. Además, cuando se encuentra una variación positiva, las modificaciones deben ser nuevamente implementadas desde cero del lado del servidor, en la aplicación original que genera el contenido, duplicando el costo de producción.

Por otro lado, en sitios web dinámicos con valor comercial y tráfico voluminoso, la arquitectura cliente-servidor se complementa con una red de servidores intermedios, ubicados cada uno entre el servidor central y un subconjunto de los clientes, llamada Red

de Entrega de Contenidos (o *Content Delivery Network* o *CDN*). Esta red guarda una copia de cada respuesta del servidor central y, ante un nuevo pedido de la misma página, entrega su propia copia, aliviando el trabajo del servidor central. Este complemento brinda una importante ganancia en performance y en descarga (u *offload*) del servidor central, pero impide variar el contenido en el servidor, puesto que una copia de una versión no debe ser entregada a un usuario al que le corresponde otra versión.

1.1. Motivación

Para el servicio de una aplicación web comercial en territorios con redes escasas y pobres, como los países emergentes, el uso de redes de entrega de contenidos es imprescindible para alcanzar una performance competitiva. En este escenario, la aplicación de la técnica de experimentación A/B variando el contenido en el servidor se vuelve imposible y deja como única opción la variación en el cliente, con serias deficiencias técnicas que van en detrimento del producto.

Algunas redes de entrega de contenidos ofrecen la capacidad de procesamiento de instrucciones, por lo que la variación en la misma red surge como una tercera opción, que resuelve los impedimentos de la primera opción y las deficiencias de la segunda. Puesto que no hay en el mercado implementaciones libres ni comerciales de herramientas de experimentación A/B que varíen el contenido en la red de entrega, se decidió encarar el desarrollo de una como núcleo del presente trabajo.

La propuesta de trabajo consiste en el desarrollo de una plataforma de experimentación A/B cuya herramienta de variación de contenido se ejecute en la red de entrega de contenidos. El servidor de aplicación puede entregar el conjunto completo de variaciones a la red de entrega y ésta distribuye las variaciones a los usuarios según sus asignaciones aleatorias particulares. Así, la mayoría de las solicitudes pueden ser satisfechas en la red, conservando la delegación de la carga, y cada usuario recibe una sola variación sin saber que está formando parte de un experimento.

1.2. Organización del trabajo

El capítulo 2 describe el problema a resolver. Se explican la técnica de experimentación A/B, cómo se exponen variaciones de contenido a los usuarios, las herramientas disponibles en el mercado, las deficiencias de las implementaciones en el cliente, las

limitaciones de las implementaciones en el servidor y las capacidades de procesamiento disponibles en la CDN.

El capítulo 3 describe la solución propuesta. Se explican los componentes en que la solución consiste, tanto para variar el contenido al que se expone el usuario como para registrar las acciones del mismo, las alternativas evaluadas en la implementacion y las iteraciones del proceso de desarrollo, y finalmente se describen todas las piezas de software desarrolladas como parte de la solución y como herramientas de asistencia.

El capítulo 4 expone los resultados obtenidos. Se describe la solución desarrollada y se destacan algunas de sus propiedades respecto del objetivo principal.

El capítulo 5 concluye el trabajo con una reflexión sobre los resultados y algunas propuestas de trabajo futuro.

Cabe mencionar que prácticamente toda la bibliografía utilizada para la confección de este trabajo se encuentra en la web. La búsqueda de bibliografía académica no fue eludida, pero sus resultados fueron casi nulos. Los lenguajes, los protocolos, las arquitecturas, el funcionamiento del software, el comportamiento de los proveedores y consumidores de internet, etc. fueron tomados de la web. El trabajo está fundamentado y todo lo citado también, sólo que con investigaciones y desarrollos de la industria, que publica en la misma web de la que aprende, indicando al lector dónde encontrar información más profunda sobre los temas específicos o la fuente de una afirmación no inmediatamente fundamentada.

2. Definición del problema

2.1. La técnica de experimentación A/B

La técnica de Experimentación A/B (o *A/B Testing*) se utiliza habitualmente en el desarrollo web para contrastar hipótesis de impacto de cambios del producto en variables de negocio. Consiste en la experimentación aleatorizada y controlada con dos variantes: la variante A o *control* es el producto original, la variante B o *tratamiento* es el producto modificado[1]. Se plantea la hipótesis de que una variación particular del producto tendrá una influencia positiva en una variable de negocio y se la expone a un subconjunto de los usuarios, mientras se mide la variable en el conjunto completo. Cuando los resultados alcanzan un umbral de confianza, llamado significatividad estadística, la hipótesis se da por comprobada o refutada según corresponda y la variación se hace permanente o se abandona [1].

La forma usual de aplicación sigue el principio *ceteris paribus* (que significa "todo lo demás igual"), según el cual se aplica una sola variación y todo lo demás permanece invariante. De esta manera, alcanzando la significatividad estadística, se puede atribuir, con el grado de confianza elegido, la variación de las variables de negocio a la variación evaluada en el producto. La técnica permite llevar a cabo experimentos simultáneos aunque, cuando las variaciones se solapan en el espacio del producto, es necesario tomar precauciones en el procedimiento para evitar dar por comprobada una hipótesis con información insuficiente [2,3]. Asimismo, en la medición de las variables de negocio, se debe tener en cuenta la distribución de las variaciones entre los usuarios para diferenciar la correlación entre los experimentos de los resultados marginales [4].

Todas las implementaciones de experimentación A/B comparten las siguientes características. Se elige una variación en el producto, una variable de negocio con la cual evaluar el producto variado, y el porcentaje de la población con la cual se va a hacer el experimento. Para cada usuario que llega al punto del producto donde tiene lugar la variación, se determina en forma aleatoria si es asignado al grupo de tratamiento o al grupo de control, y se le entrega la versión correspondiente del producto. La correspondencia

[1] En realidad el método es válido con cualquier número de variantes y su forma general se llama *Método de Contraste de Hipótesis*. Es en el marketing del método mismo que se lo ha simplificado a dos variantes para facilitar su comprensión a operadores no familiarizados con los conceptos estadísticos subyacentes.

entre ese usuario particular y su asignación se almacena para entregarle la misma versión del producto en visitas posteriores y que su experiencia sea consistente. En todos los casos se evalúa la variable de negocio y, acumulada una cantidad suficiente de visitas, la comprobación o refutación de la hipótesis se hace observando su valor en cada uno de los grupos.

2.2. Herramientas de experimentación A/B

Existen ya productos en el mercado que implementan la técnica de experimentación A/B y que pueden ser integrados para aplicar la técnica sin un desarrollo particular. Algunas implementaciones funcionan en el lado servidor de la arquitectura web, tanto como bibliotecas de funciones para lenguajes particulares como en forma de servicios que otros componentes pueden consultar, y se encargan de automáticamente seleccionar las variaciones definidas en la misma aplicación servidor. Otras funcionan en el lado cliente de la arquitectura web, como guiones de instrucciones que se integran en los documentos HTML del producto, y aplican las variaciones sobre el documento original una vez generado y recibido por el usuario.

La siguiente tabla muestra ejemplos de implementaciones de experimentación A/B disponibles en el mercado.

Tabla 1. Implementaciones de experimentación A/B disponibles en el mercado.

	Del lado del servidor	Del lado del cliente
Como bibliotecas de funciones de PHP o Javascript	php A/B test: http://phpabtest.com/ phpScenario: http://phpscenario.org/ kumite: https://github.com/lwc/kumite php-redis A/B testing: https://github.com/aliasaria/PHP-redis-A-B-Testing	ABBA: http://blog.alexmaccaw.com/abba
Como servicios	SixPack: http://chairnerd.seatgeek.com/sixpack/	Optimizely: https://www.optimizely.com/ Convert: http://www.convert.com/ Unbounce: http://unbounce.com/ Visual Website Optimizer: https://vwo.com/

A continuación se detalla el análisis de estas opciones y la experiencia que se tuvo con ellas y que motivó el presente trabajo.

2.3. Variación de contenido en el servidor

2.3.1. El tiempo de transferencia de la respuesta

La performance de una aplicación web, desde el punto de vista del usuario, se mide habitualmente con los tiempos que transcurren entre una acción de este y la respuesta de aquella [14]. Una métrica muy usada es el tiempo de transferencia de los objetos que conforman la aplicación, que se descompone en la suma del producto del tamaño del objeto por la velocidad de transferencia, más el tiempo que transcurre desde que se emite el pedido hasta que llega la primera unidad de información, o primer *byte*.

Tiempo transferencia = tamaño x velocidad + tiempo primer byte (1)

Los tres componentes de esta métrica pueden ser optimizados. Los objetos son habitualmente comprimidos en el servidor y descomprimidos en el cliente, para minimizar el tamaño de la transferencia [15]. La velocidad depende del ancho de banda disponible en todos los enlaces del camino entre el servidor y el cliente, y es habitual que el enlace más lento sea el último, el que une al cliente con su Proveedor de Servicio de Internet (o *Internet Service Provider* o *ISP*), y que su optimización esté fuera del alcance del productor. Finalmente, el tiempo hasta el primer *byte* es la suma del tiempo que la aplicación tarda en generar la respuesta más el tiempo que tardan el pedido y el primer paquete de la respuesta en viajar por la red que la comunica con el cliente [16]. Por lo tanto, cuanto menor sea la distancia de red entre ellos, cuanto menos saltos haya y menos reenvíos de paquetes ocurran, menor será el tiempo hasta el primer *byte*.

Tiempo primer byte = latencia pedido + tiempo generación + latencia respuesta (2)

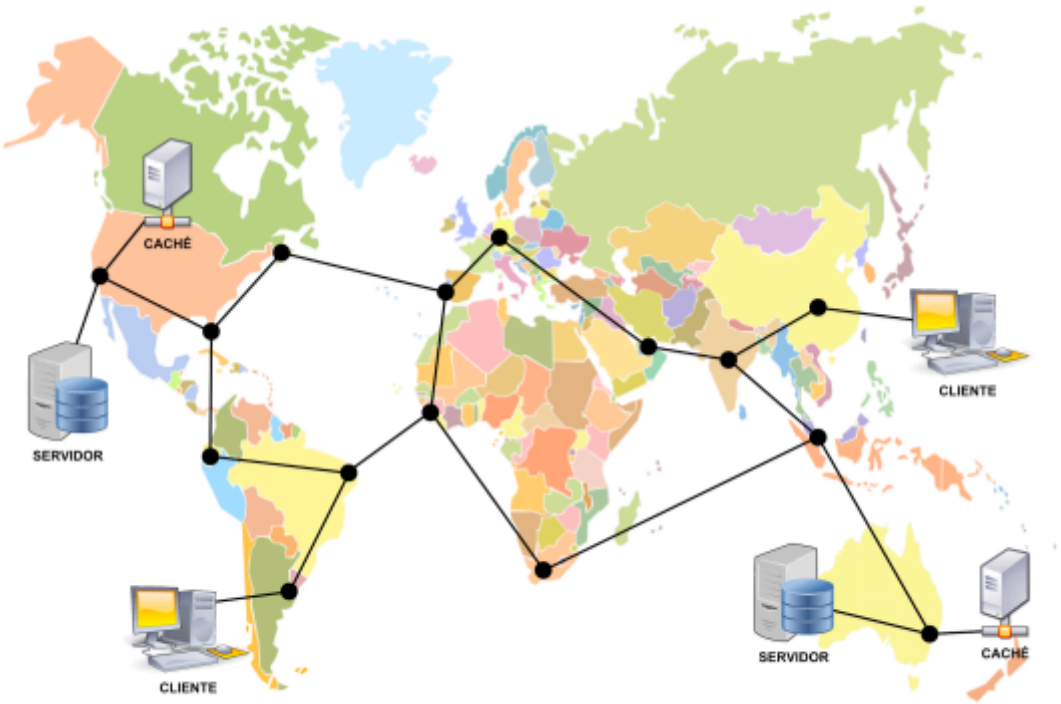
Además de las motivaciones técnicas que se puedan tener para minimizar el tiempo al primer *byte*, hay una fuerte razón comercial para hacerlo: el tiempo que transcurre entre que el usuario hace el pedido y que algo comienza a aparecer en su pantalla está directamente relacionado con la tasa de abandono del sitio web [11].

2.3.2. El tiempo de generación de la respuesta

Una manera de disminuir el tiempo promedio de generación de la respuesta es generar una sola vez la respuesta a cada pedido, guardar una copia del resultado y entregar la copia en las siguientes ocurrencias del mismo pedido. Para que una respuesta sea reutilizable, es necesario que sea válida para más de un usuario: no debe ser un objeto privado de un usuario particular ni, siendo público, estar personalizado.

Esta técnica no es nueva: es *caching* aplicado a objetos web [22]. El *caching* consiste en guardar una copia reutilizable del resultado de una operación compleja en un lugar desde donde se pueda recuperar con un costo menor que ejecutar nuevamente la operación. En el caso de la web, la *caché* guarda una copia de los documentos HTML, guiones Javascript, hojas de estilo y recursos multimedia que componen las páginas web para entregarla en pedidos subsiguientes (figura 1).

Figura 1. Esquema de una red global de conmutación de paquetes con clientes, servidores y cachés. Las cachés sustituyen a los servidores respondiendo con copias de respuestas anteriores mientras aún son válidas.



Un beneficio adicional asociado a esta técnica es la descarga de trabajo en los servidores de aplicación. La generación de una respuesta a partir de su materia prima (estructuras de

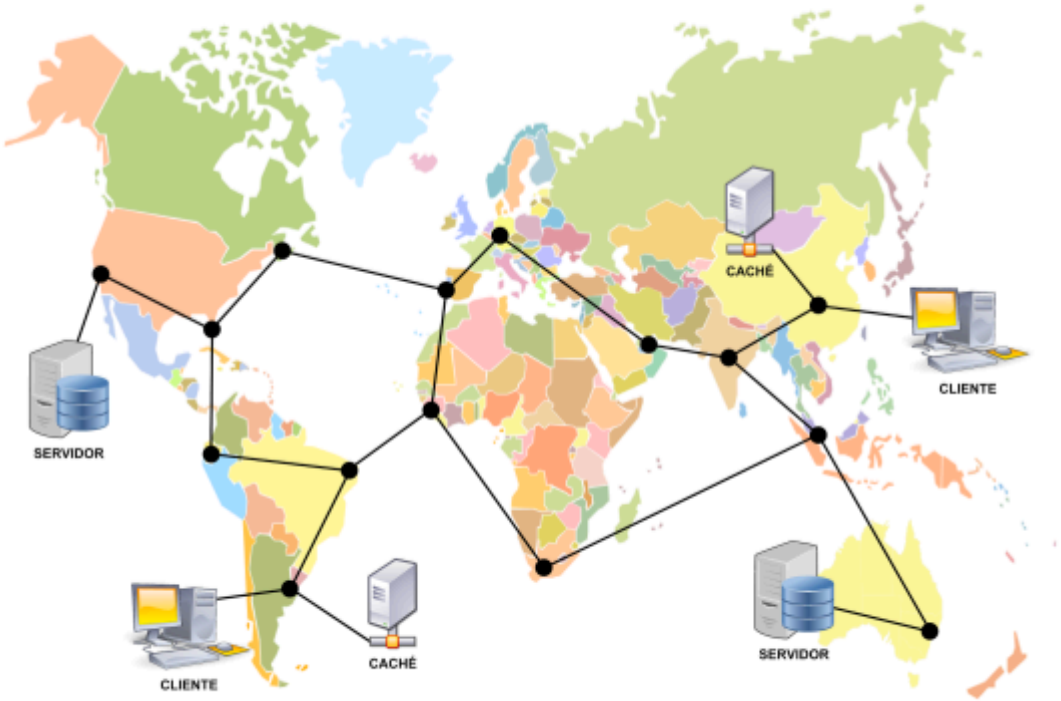
datos y plantillas de presentación) es un trabajo más costoso que la mera lectura de un objeto binario almacenado localmente, que es usualmente de costo O(1) en búsqueda y O(n) en transferencia. Por cada reutilización de una respuesta, la suma total de trabajo ha sido disminuida en proporción a esa diferencia.

2.3.3. La latencia de la transferencia

El servidor de aplicación suele estar ubicado en un solo punto geográfico de la Internet. No es infrecuente tener servidores en más de un punto pero, si la aplicación tiene estado interno, sincronizar las transacciones a través de enlaces no locales es costoso (técnica y económicamente) y sólo se hace por cuestiones legales o de disponibilidad, nunca de performance. Una forma de disminuir la latencia de la transferencia es reducir la distancia de red entre el servidor y el cliente pero, cuando los clientes están geográficamente dispersos (siempre en términos de red), es imposible encontrar un punto que sea cercano a todos.

Lo que sí se puede mover en estos casos es los servidores de *caching*. En lugar de ubicarlos inmediatamente al frente de los servidores de aplicación, se los ubica en medio de los focos de clientes, en los centros neurálgicos de la Internet. La distancia al servidor de

Figura 2. Esquema de una red global de conmutación de paquetes con clientes, servidores y cachés. Puesto que la frecuencia de comunicación entre clientes y cachés es igual o mayor que aquella entre cachés y servidores, ubicar las cachés cerca de los clientes disminuye el tiempo promedio de respuesta.

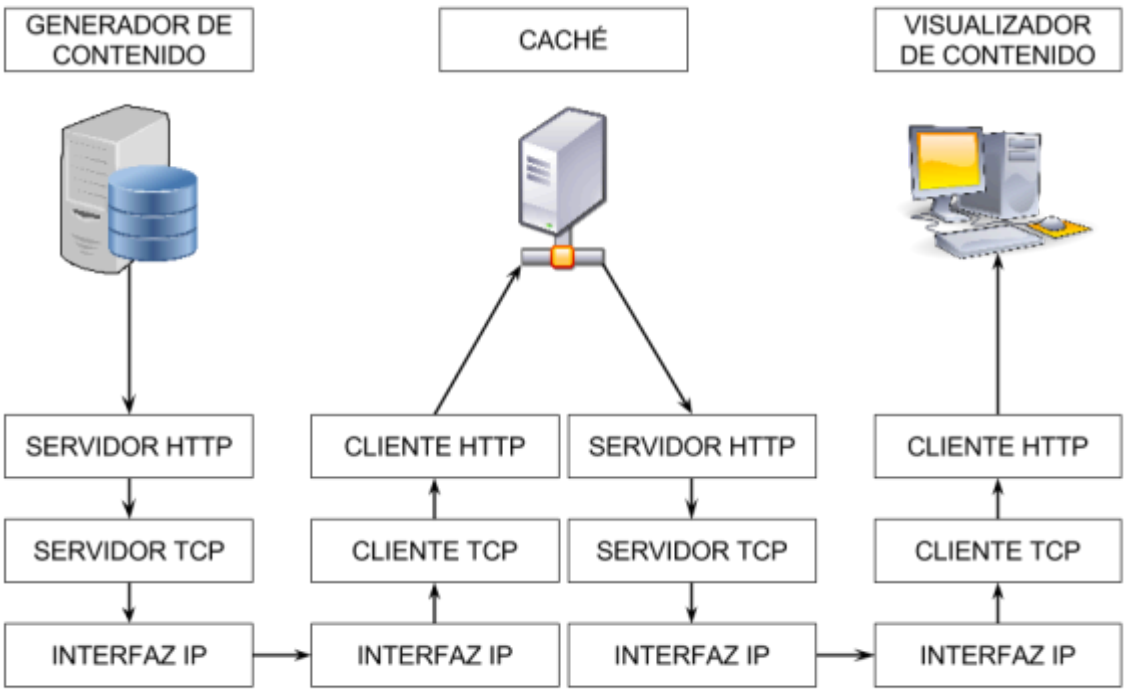


aplicación y el tiempo de respuesta de pedidos nuevos no cambian, pero la distancia al servidor de *caching* y el tiempo de respuesta para pedidos cuya respuesta está en caché disminuye notablemente (figura 2).

Una mejora adicional a esta técnica consiste en tener los servidores de *caching* comunicados entre sí a una corta distancia de red y que cada uno, cuando no tiene una respuesta propia a un pedido, intente conseguirla primero en su par más cercano antes de recurrir al origen. El costo es mayor cuando no se encuentra el objeto, pero el aumento es sólo una operación de complejidad O(1), y la probabilidad de no encontrar el objeto en caché es notablemente menor, más precisamente el cuadrado de la original. Esta red distribuida de servidores de *caching* se llama *Red de Entrega de Contenidos* (*Content Delivery Network* o *CDN* en inglés) [17].

Rara vez una CDN necesita para funcionar más información que la ubicación del servidor de aplicación. Si bien trabaja en la capa de aplicación de la pila de red, en una subdivisión de esa capa será la subcapa inferior, igual que un servidor HTTP, y para la aplicación web no será más que infraestructura. Por eso, es frecuente que las redes de entrega de contenidos sean provistas por terceros, pues tienen en sí mismas una gran complejidad que escapa

Figura 3. Pilas de protocolos de servidor, caché y cliente en una comunicación HTTP intermediada por una CDN.



habitualmente el ámbito de los proveedores de servicios de aplicación. Ejemplos de redes de entrega de contenidos son Akamai Edge [18] y Amazon Cloudfront [23].

La figura 3 muestra la parte superior de las pilas de red y aplicación de los tres integrantes de esta arquitectura Cliente-CDN-Servidor.

2.3.4. La experimentación A/B con una CDN

Para ser transparente ante el cliente, la clave primaria que la CDN usa para el *caching* es la misma que usa el cliente para pedir los objetos: el localizador uniforme de recurso (o *Uniform Resource Locator* o *URL*), la dirección estandarizada para ubicar recursos en la web, que incluye el protocolo, nombre de dominio, ruta, recurso y parámetros.

Un experimento A/B consiste en responder versiones diferentes de un mismo objeto ante pedidos equivalentes, es decir, en el mismo URL. Si la decisión de cuál versión responder se toma en el origen, la CDN almacenará la versión que se asigne arbitrariamente a quien haga el primer pedido, y devolverá la misma en todos los pedidos subsiguientes, inutilizando el estudio. Sólo es posible llevarlo a cabo indicando a la CDN que los objetos variados *no son reutilizables*, pero entonces se perderá la ganancia que esta provee y el tiempo de transferencia crecerá a su peor caso: el del camino completo hasta el origen.

Es común en negocios globales de la Internet tener un sitio separado por cada país, con funcionalidad equivalente, pero conjuntos de datos estancos y excluyentes. Una alternativa que se utilizó mucho en los inicios de la experimentación fue usar poblaciones estáticas: en lugar de hacer una selección al azar para cada usuario y asignarlo a la población de tratamiento o la de control, se probaba el tratamiento en uno o más países. Fue un progreso al comienzo, comparado con hacer cambios globales y hacer las mediciones de las variables relevantes antes y después de cada cambio, pero con el tiempo se descubrió que las diferencias culturales entre usuarios de distintos países eran más significativas de lo esperado, haciendo que los resultados fueran menos confiables (porque las poblaciones comparadas no eran equivalentes) y menos útiles (porque lo que funcionara bien en un país no necesariamente lo haría en otros). Era el momento de cambiar de estrategia.

2.4. Variación de contenido en el cliente

La alternativa fue pasar la decisión de qué versión de un objeto exponer al usuario al otro lado de la CDN: el cliente. Se entregaría la versión de control de cada página, acompañada por un script que genera un valor al azar y, si el resultado lo indica, varía el contenido en el mismo cliente. El código fuente de la página, lo que el cliente recibe, sería entonces constante y, por lo tanto, *cacheable* en todo el camino, con lo cual la CDN podría ser utilizada sin inconvenientes. Se encontraron, sin embargo, otras limitaciones, que se explican a continuación.

2.4.1. Flickering

El contenido de una página se define con un documento HTML. El documento HTML es básicamente una descripción declarativa de elementos visuales a presentar en la pantalla, pero se puede asociar comportamiento a estos elementos usando guionado o *scripting*, una programación interpretada de alto nivel cuyos elementos atómicos son acciones específicas de la plataforma [19]. Un experimento A/B que funciona en el cliente se implementa definiendo declarativamente en el documento la versión de control, la versión no modificada, y guionando la modificación del contenido ante la detección de que se debe mostrar una versión distinta (figura 4).

Los navegadores web dividen la tarea de presentación de una página entre varios hilos de ejecución, entre los cuales se encuentran, por separado, uno que parsea el documento HTML recibido del servidor y construye un modelo de objetos del documento (*Document Object Model* o DOM), y otro que lee periódicamente el DOM y dibuja el documento en la pantalla (*rendering*) [7]. Estos dos hilos funcionan en *pipeline*: no se espera a que el DOM esté construido completamente para dibujar en la pantalla. El DOM es un árbol y, no bien hay nodos terminados, son puestos a disposición del hilo de rendering.

Figura 4. Variación de contenido en el cliente. El contenido original del documento es la versión de control y un script sujeto a la condición de tratamiento cambiar el contenido tan pronto el documento es modificable. Se destacan el guion de modificación y el contenido a ser modificado.

```
<html>
  <head>
    <script>
```

```
      if (tratamiento) {

        function tratar() {
          var titulo = document.getElementById('titulo');
          titulo.innerHTML = 'Avisos clasificados';
        }

        document.addEventListener('DOMContentLoaded', tratar)

      }

    </script>
  </head>
  <body>
    <h1 id="titulo">Anuncios Clasificados</h1>
  </body>
</html>
```

El contenido de la página se modifica haciendo cambios en el DOM, que son posteriormente reflejados en la pantalla por el hilo de rendering. El redibujado no ocurre habitualmente más que una vez cada 40-50 milisegundos, frecuencia suficiente para que el ojo humano perciba los cambios como continuos en el tiempo. Cuando un elemento del DOM es modificado, queda marcado para redibujar en la próxima pasada del hilo de rendering. A su vez, este dibuja cada elemento en el estado en que lo encuentra al pasar. Si una variable de un elemento cambia dos veces en un mismo período, el redibujado sólo verá su último estado. Si un elemento es modificado en el mismo período en que entra en el DOM, su versión original nunca será dibujada.

Entonces, para hacer el experimento A/B, el objetivo es aplicar las variaciones en el documento original lo antes posible. Para modificar cada elemento que forme parte de la variación, es necesario que ya exista en el DOM, por lo que lo ideal sería sincronizar con alguno de los hilos mencionados.

La única primitiva de sincronización que hay con esos hilos es la suscripción al evento `DOMContentLoaded`, que indica que el DOM se ha terminado de construir, pero el rendering comienza antes. La frecuencia del rendering es suficientemente alta para que cambios progresivos sean percibidos como continuos, pero suficientemente baja para no poder hacer cambios absolutamente imperceptibles, por lo que un solo período es tiempo suficiente para que algo sea visto por el usuario. Si la versión original del documento es dibujada, aún cuando la variación se dibuje en el período inmediatamente posterior, el usuario ya verá el

cambio. Esta visualización momentánea de una versión distinta del documento se llama *flickering*, y es un efecto no deseado, pues provoca en el usuario una experiencia negativa.

Sin embargo, mostrar la versión original por una fracción de segundo es el mejor caso de *flickering*. Javascript tiene la primitiva `document.write()`, que permite escribir directamente en el stream de *entrada* del DOM, por lo que la ejecución de los scripts debe ser, por definición, *sincrónica*: durante la ejecución de un script, la construcción del DOM se detiene. Puesto que los scripts pueden tener su código fuente en un recurso externo, como las hojas de estilo y las imágenes, cuando una referencia a un script externo aparece en el documento, se hace el pedido del recurso y, durante un tiempo bastante mayor al período del rendering, la construcción del DOM está detenida y todo lo dibujado es claramente visible por el usuario. A veces, sobre todo en redes lentas, el contenido dibujado es visible durante varios segundos antes de que el evento `DOMContentLoaded` ocurra (figura 5).

Luego, no hay que esperar a que el DOM esté completo: hay que llegar antes, idealmente en el mismo período de rendering en el que el elemento a modificar entró en el DOM. Pero no hay ninguna primitiva de sincronización, por lo que la única forma es hacer una espera activa (o *polling*) a que el elemento buscado aparezca.

Figura 5. Retraso de la aplicación del tratamiento. El contenido original se muestra inmediatamente, pero el tratamiento se aplica sólo después de la carga del recurso externo, cuya demora es usualmente mayor que el período de *rendering*. Se destacan la declaración del recurso externo y la sujeción de la ejecución al evento de carga.

```
<html>
  <head>
    <script>
      if (tratamiento) {

        function tratar() {
          var titulo = document.getElementById('titulo');
          titulo.innerHTML = 'Avisos clasificados';
        }

        document.addEventListener('DOMContentLoaded', tratar)

      }
    </script>
  </head>
  <body>

    <h1 id="titulo">Anuncios Clasificados</h1>

    <script src="//js/recurso_externo.js"></script>
```

```
  </body>
</html>
```

Los elementos del DOM son objetos y para modificarlos se necesita tener referencias físicas a ellos. El camino más crudo es saltar de referencia en referencia desde la raíz del árbol de objetos pero, además de que las primitivas son de un nivel muy bajo para una tarea de este tipo, el código queda acoplado a la forma física del árbol. Una alternativa es buscar los elementos con una declaración *XPath*, más simple y de más alto nivel, pero aún acoplada a la forma física del documento. La forma más común, de más alto nivel y desacoplada de la forma física, es usar selectores, en los cuales se definen los elementos buscados por comprensión, por su identificación, clase y tipo lógicos. El problema de buscar los elementos en el DOM mientras no está completo, definiendo por comprensión el conjunto de lo que se busca, es que no hay forma de saber si se han encontrado todos los elementos.

Supongamos que se quiere modificar todos los enlaces del documento y quitarles el subrayado. El selector para todos los enlaces sería "A" (el elemento HTML usado para construir enlaces). Si hacemos una búsqueda activa durante la construcción del DOM, en la primera pasada en la que se encuentre *algún* enlace, la búsqueda del selector se cumplirá, se ejecutará la acción asociada de quitar el subrayado, y finalizará la búsqueda. Si más adelante en la construcción aparecen nuevos enlaces, no se efectuará ningún cambio sobre ellos (figura 6).

Figura 6. Espera activa hasta la aparición del objetivo. Tan pronto hay una coincidencia con la búsqueda, se aplica la variación y finaliza la espera. Si, más tarde, llegan nuevos elementos que coinciden con la búsqueda, no resultan variados. Se destacan la función con reentrada diferida y el recurso externo que demora la aparición de los ultimos elementos.

```
<html>
  <head>
    <script>
      if (tratamiento) {
        function tratar() {

          var ps = document.getElementsByTagName('p');

          // si se encontraron elementos
          if (ps.length > 0) {
```

```
            // tratar los elementos encontrados
            for (var i = 0; i < ps.length; i++) {
              ps[i].setAttribute('align', 'right');
            }

          } else {

            // intentar nuevamente en 50 milisegundos
            setTimeout(tratar, 50);

          }

        }
        tratar();
      }
    </script>
  </head>
  <body>

    <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do
eiusmod tempor incididunt ut labore et dolore magna aliqua.</p>

    <p>Ut enim ad minim veniam, quis nostrud exercitation ullamco
laboris nisi ut aliquip ex ea commodo consequat.</p>

    <script src="//js/recurso_externo.js"></script>

    <p>Duis aute irure dolor in reprehenderit in voluptate velit esse
cillum dolore eu fugiat nulla pariatur.</p>

    <p>Excepteur sint occaecat cupidatat non proident, sunt in culpa qui
officia deserunt mollit anim id est laborum.</p>

  </body>
</html>
```

Para evitar esto y cubrir los casos que puedan aparecer más tarde, se podría continuar la búsqueda hasta que el DOM esté completo. En cada pasada de la búsqueda activa, se hace la búsqueda de enlaces, se toman todos los que se encuentren y se les configura el subrayado como apagado. Si ya estaba apagado, queda apagado.

El problema ocurre ahora cuando el cambio no es idempotente. Por ejemplo, si se quiere buscar todas las ocurrencias de texto en cursiva y encerrarlas entre comillas. El selector a buscar sería "EM" (el elemento HTML usado para indicar énfasis). Se buscan los elementos EM y se reemplaza en cada uno el contenido por comillas + contenido + comillas. Pero si se

vuelve a buscar periódicamente para encontrar nuevos elementos EM, los ya modificados serán nuevamente encontrados, y se volverá a agregarles comillas (figura 7).

Figura 7. Espera activa hasta que el documento esté completo, pero con una transformación *no idempotente*. En este caso, cada elemento EM es envuelto entre comillas tantas veces como iteraciones ocurran, en lugar de sólo una vez. Se destacan la rutina de modificación de elementos y los elementos a modificar.

```
<html>
  <head>
    <script>
      if (tratamiento) {
        function tratar() {
          var ems = document.getElementsByTagName('em');
          if (ems.length > 0) {

            // agregar comillas a cada EM encontrado
            for (var i = 0; i < ems.length; i++) {
              ems[i].innerText = '"' + ems[i].innerText + '"';
            }

          } else {
            setTimeout(tratar, 50);
          }
        }
        tratar();
      }
    </script>
  </head>
  <body>
    <p>Lorem ipsum dolor sit amet, <em>consectetur</em> adipiscing elit,
sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.</p>
    <p>Ut enim ad minim veniam, quis nostrud exercitation ullamco
laboris nisi ut aliquip ex ea commodo consequat.</p>
    <p>Duis aute irure dolor in reprehenderit in <em>voluptate</em>
velit esse cillum dolore eu fugiat nulla pariatur.</p>
    <p>Excepteur sint occaecat cupidatat non proident, sunt in culpa qui
officia <em>deserunt</em> mollit anim id est laborum.</p>
  </body>
</html>
```

Una forma de evitar esto sería construir un catálogo de los elementos ya modificados y, en cada pasada, buscar cada elemento encontrado en el catálogo y, sólo si no se encuentra, modificarlo y agregarlo.

Lamentablemente, aún pagando el precio de agregar una búsqueda activa y una estructura de control auxiliar, que incrementan la carga de procesamiento y la demora de construcción de la página, no se resuelve el problema. Sin una forma de capturar el evento de agregado de un elemento al DOM o de inicio de la pasada de rendering, no hay garantía de llegar a tiempo a modificar los elementos del DOM antes de que sean dibujados por primera vez. Una *condición de carrera* inherente nos obliga a decidir entre demorar la carga de la página por aumentar la frecuencia de la búsqueda activa, y arriesgarse al flickering en el experimento por disminuirla.

2.4.2. Doble implementación

Otra desventaja que se encontró en la implementación de la experimentación A/B como modificación del documento original recibido por el cliente es que, una vez que un experimento termina y la versión de tratamiento ha obtenido un mejor resultado que la versión de control, la variación del producto debe ser implementada *nuevamente* sobre el producto mismo. Cuando se hace en el servidor, se implementa la variación y se descarta por perder o queda por ganar. Cuando se hace en el cliente, se implementa la variación y se descarta por perder o se descarta igual y se construye de nuevo en el servidor. Esto no sería un problema si la implementación del cambio en el cliente tuviese un costo menor que la implementación en el producto mismo y la hipótesis fuese más probable de refutar que de comprobar, pero rara vez es este el caso.

Por un lado, los cambios en el servidor son generalmente declaraciones de la forma final del contenido. Los cambios en el cliente, en cambio, deben implementarse necesariamente como acciones, como modificaciones individuales de valores de los parámetros que definen el contenido, además de agregar lo necesario para ubicar estos cambios en el tiempo (sincronización, búsqueda activa, lo que fuere). El código será ejecutado en plataformas y navegadores distintos, y no es infrecuente tener que ramificar el código para implementar más de una vez alguna funcionalidad en la que no son compatibles. Las implementaciones son en la mayoría de los casos en lenguajes distintos y programadas por diferentes desarrolladores, sumando al proceso una instancia de comunicación y traducción que introduce más riesgo de errores, y engrosando la lista de habilidades requeridas en el equipo de trabajo. Finalmente, las declaraciones de las variaciones en el servidor se hacen en el código fuente mismo, y el desarrollador sabe en todo momento que su código está sujeto a ellas, mientras que en el lado del cliente se hacen por separado, y quien desarrolla

el contenido puede no saber que su código está condicionado y confiar en una ejecución que podría no ocurrir.

Por otro lado, los tratamientos no son cambios azarosos que se prueban con la esperanza de que funcionen: son hipótesis fundadas sobre el resultado de mediciones previas del comportamiento del usuario, de su circulación por el espacio de la página, de sus acciones sobre ella, de sus elecciones ante diferentes opciones. Siempre hay un componente de incertidumbre, que es el que se quiere eliminar con el experimento, pero cada experimento tiene un costo, y debe ser correspondido con una hipótesis que se presuma fundadamente más probable de comprobar que de refutar.

2.5. Variación de contenido en la CDN

2.5.1. Ejecución de código en la CDN

Edge Side Includes, o ESI, es un lenguaje de marcado para ensamblado de contenido web dinámico, diseñado para ser interpretado en sistemas de *caching* [6]. Su operación principal, `include`, permite ordenar a la CDN que, en un punto específico de un documento, incluya el contenido de otro documento. El cliente hace un solo pedido a la CDN, la CDN hace dos pedidos al origen (uno del documento incluyente, otro del documento incluido), compone el documento final y lo entrega al cliente.

Supongamos que un documento contiene dos elementos, uno costoso de generar pero que sólo cambia una vez por día, otro más barato de generar pero que cambiar cada un minuto. Generar el documento completo tiene el costo alto del primer elemento, y sólo es reutilizable durante un minuto, pues en el siguiente minuto el segundo elemento habrá cambiado. Si se reemplaza el documento por dos documentos, uno con el contenido costoso y una referencia al otro documento, el otro con sólo el segundo elemento, el primer documento podrá ser construido una sola vez por día, el segundo cada un minuto, y la red de entrega podrá ensamblar los contenidos de los dos en un único documento ante cada pedido, e incluso cachear también el resultado.

La inclusión es la operación básica de ESI, de la que toma su nombre, citando por su similitud a las ya antiguas *Server Side Includes* [24], pero cuenta con variables,

instrucciones de iteración y selección y control de excepciones, que permiten condicionar o repetir las inclusiones y además crear nuevas lógicas que trasciendan la intención inicial.

Si bien ESI existe hace más de una década, aún no ha sido estandarizado. Sin embargo, algunos de sus propulsores lo ofrecen hace tiempo a sus clientes, en versiones propietarias. Akamai, por ejemplo, ofrece como valor adicional información de introspección de la CDN, como la ubicación geográfica del cliente de un pedido, métricas de calidad de la red en la que este se encuentra o capacidades del dispositivo que está usando, expuesta en variables globales para el lenguaje [25]. Un ejemplo de uso de esta información es el condicionamiento de un bloque de código a si el usuario accede desde un dispositivo móvil o no. Un usuario con un cliente de escritorio, más veloz y potente y con una pantalla más grande, puede recibir un documento completo con un artículo principal y recomendaciones en un lado. Un usuario con un cliente móvil, más lento y con una pantalla más pequeña, puede recibir sólo el artículo principal, sin que el servidor central deba generar una versión particular para él: es la CDN quien elimina el bloque del documento antes de entregarlo.

2.5.2. Propuesta de trabajo

La propuesta de este trabajo consiste en el desarrollo de una herramienta de experimentación A/B que efectúe la variación del contenido en la CDN. En cada URL, el servidor deberá entregar un documento que incluya todas las variaciones a experimentar, envueltas en el código ESI, el cual hará que la decisión de cuál usar en cada caso se tome en la CDN. El documento será entonces constante, y podrá ser guardado en la CDN para ser reutilizado sin solicitar una nueva generación. Cada cliente, a su vez, recibirá un documento estático, con las variaciones que le correspondan según la distribución del experimento, ya aplicadas. Para el servidor, la variación se hace más tarde, con las ventajas de las implementaciones en el lado del cliente. Para el cliente, la variación se hace más temprano, con las ventajas de las implementaciones en el lado del servidor.

3. Desarrollo de la propuesta

En la primera sección de este capítulo se detallan las partes que componen la solución al problema: un componente para aplicar las variaciones al contenido expuesto al usuario y otro componente para registrar las acciones de los usuarios expuestos a cada variación.

En una segunda sección, se explican las etapas del desarrollo iterativo efectuado para llegar a ella, cada una con su justificación y su posterior análisis. Las principales cuestiones fueron si generar páginas separadas para cada versión de un experimento o si variar el contenido en el interior de un documento único, si generar el contenido variado en una pasada o en dos pasadas, y si marcar cada registro de acción del usuario con información sobre el usuario o sobre el experimento.

En una última sección, se listan todas las piezas de software desarrolladas, cada una con una breve explicación de su composición y su rol en el desarrollo del trabajo.

3.1. Componentes

3.1.1. Variación del contenido

El componente desarrollado específicamente para este trabajo, y que constituye su principal aporte, es el variador de contenido. Es un módulo de funciones de PHP, lenguaje utilizado en el sitio web de OLX [13], que permite al desarrollador definir un conjunto de experimentos y las variaciones que en ellos se medirán, y hace las modificaciones correspondientes en la salida para que las variaciones sean distribuidas entre los usuarios y las acciones de estos últimos registradas.

En la aplicación actual ya se usa código ESI, pero está literalmente entrelazado con el HTML y con redundancia, por lo que una abstracción es ya necesaria hoy para reducir retrabajo y riesgo de errores. Tal abstracción fue construida también en el marco de este trabajo, para contar con ella como infraestructura para el módulo de variación. Es un módulo adicional que abstrae las estructuras principales del lenguaje (asignación de variables, iteración, selección, emisión de salida) en funciones de PHP, liberando al programador de la revisión sintáctica y gramatical, y permitiéndole definir sólo lo que le concierne: la lógica de aplicación delegada a la CDN.

Un documento básico en ESI, en el que no se solicita ninguna modificación a la CDN, consiste en una sola instrucción de emisión de salida literal (`esi:text`), que imprime el documento HTML completo en la salida. Las acciones que se delegan a la CDN se entrelazan con esta instrucción como en el ejemplo de la figura 8.

Figura 8. Ejemplo de documento ESI, que produce un documento HTML dinámico, dependiente de la presencia de una cookie. Se destaca el código ESI que envuelve los bloques de contenido.

```
// bloque de contenido literal
<esi:text>
<html>
  <head>
    <title>Example Page</title>
  </head>
  <body>
    <p>
</esi:text>

<esi:choose>
  <esi:when test="$exists($(HTTP_COOKIE{'username'}))">

    // cuando hay una sesión, saludar al usuario
    <esi:text>
      Bienvenido $(HTTP_COOKIE{'username'})
    </esi:text>

  </esi:when>
  <esi:otherwise>

    // de lo contrario, ofrecer iniciar sesión
    <esi:text>
      <a href="/login">Iniciar sesión</a>
    </esi:text>

  </esi:otherwise>
</esi:choose>

// otro bloque de contenido literal
<esi:text>
    <p>
  </body>
</html>
</esi:text>
```

En el ejemplo, el contenido emitido por el origen es constante, y puede ser reutilizado, mientras que el contenido que recibe el cliente está ajustado a su situación particular. La

evaluación del caso particular del cliente ocurre en la CDN, el punto a la vez externo al cliente y menos distante de toda la arquitectura. El módulo de variación de la herramienta de experimentación A/B funciona sobre esta infraestructura: la salida del servidor central es un documento ESI.

El módulo recibe como entrada inicial una lista de experimentos, cada uno con un nombre y un conjunto de variaciones, y cada una de estas con un nombre y la asignación de una parte del tráfico. En cada respuesta, lo primero que hace es decidir qué variación será presentada al cliente con una función aleatoria con la distribución definida. En la jerga de experimentación A/B, a ejecutar esta función se le llama *tirar el dado*. Para no alterar la experiencia de uso, cada usuario debe recibir siempre la misma variación de cada experimento. Por ello, luego de asignar una variación, el resultado es almacenado en el cliente. En las visitas posteriores del mismo cliente, en lugar de ejecutar nuevamente la función aleatoria, se usa el valor almacenado, o *dado*. El bloque de la figura 9 muestra una versión reducida del código de decisión, con una función de lectura del dado y una función de selección de la variación de un experimento.

Figura 9. Tirada y almacenamiento del dado y selección de la variación de un experimento, implementadas en ESI. Se destacan los bloques más importantes de la lógica.

```
// función que tira el dado y retorna su valor

<esi:function name="tirarDado">
  <esi:choose>

    // si existe una cookie con el valor del dado, tomarla
    <esi:when test="$exists($(HTTP_COOKIE{'dado'}))">
      <esi:assign name="dado" value="$int($(HTTP_COOKIE{'dado'}))"/>
    </esi:when>

    // si no existe, crearla
    <esi:otherwise>
      <esi:assign name="dado" value="$rand(100)"/>
      $set_cookie('dado', $(dado))
    </esi:otherwise>

  </esi:choose>
  <esi:return value="$(dado)"/>
</esi:function>

// función que retorna la variación según el dado y el % definido

<esi:function name="elegirVariacion" args="['dado','porcentaje']">
```

```
  <esi:choose>

    // si el dado cae entre 0 y $(porcentaje), es tratamiento o A
    <esi:when test="$(dado) >= 0 and $(dado) < $(porcentaje)">
      <esi:assign name="variacion" value="'TRATAMIENTO'"/>
    </esi:when>

    // de lo contrario, es control o B
    <esi:otherwise>
      <esi:assign name="variacion" value="'CONTROL'"/>
    </esi:otherwise>

  </esi:choose>
  <esi:return value="$(variacion)"/>
</esi:function>

// ejecución al inicio de la página

<esi:assign name="dado" value="$tirarDado()"/>
<esi:assign name="var{'exp1'}" value="$elegirVariacion($(dado), 50)"/>
```

Con el módulo instalado y configurado, luego, en la definición del contenido de los documentos de salida de la aplicación, se pueden asignar bloques a variaciones particulares. Todos los bloques estarán presentes en la salida de la aplicación, pero condicionados en ESI a que el dado tenga un valor dentro del rango asignado a la variación, con lo cual la selección ocurre recién en la CDN. El bloque de la figura 10 muestra una versión resumida de la lógica.

Figura 10. Selección de contenido a emitir a la salida en función de la variación seleccionada para un experimento. Se destaca el código ESI que envuelve los bloques de contenido.

```
<esi:choose>
  <esi:when test="$(variacion{'exp1'}) == 'A'">
    <esi:text>

      <!-- Versión A -->
      <b>Publicá tu anuncio</p>

    </esi:text>
  </esi:when>
  <esi:otherwise>
    <esi:text>

      <!-- Versión B -->
      <b>Publicá tu aviso</p>
```

```
    </esi:text>
  </esi:otherwise>
</esi:choose>
```

3.1.2. Tracking

El objetivo de un experimento es saber cuál variación produjo un mejor resultado en una o más variables de negocio, para conocer las cuales hay que *medir*. La medición se hace contando el número de usuarios que han sido expuestos a cada variación y, entre ellos, el número que ejecuta alguna de las acciones en las que el negocio tiene un interés. La ejecución de una de tales acciones por parte del usuario es llamada una *conversión* y, generalizando, la cantidad de conversiones sobre el total de exposiciones de una variación es el *porcentaje de conversión* de la variación. La mejor variación será la que tenga el mayor porcentaje de conversión.

Estas mediciones se enmarcan en la tarea conocida como análisis de la web (o *Web Analytics*) [20], para la cual hay muchas herramientas muy buenas en el mercado. En OLX se usa Google Analytics [21], una herramienta de Google, y el equipo está satisfecho con ella, por lo que integrar el tracking de los experimentos A/B en ella es doblemente positivo. Por un lado, la nueva tarea de medición queda integrada con las otras tareas de analytics. Por otro, todo lo que Google Analytics provee se puede aprovechar para la nueva herramienta, ahorrando costos de desarrollo (en particular, la interfaz de usuario, que suele ser muy compleja) y de funcionamiento.

Para contar las exposiciones de las variaciones y sus conversiones, es necesario hacer nuevos pedidos desde los clientes, y esos pedidos no pueden ser atendidos por la CDN, porque deben ser registrados en una base de datos, con la que la CDN no cuenta. La delegación de lógica a la CDN tiene como uno de sus principales objetivos liberar de tráfico al origen, aumentando la cantidad de veces que una página puede ser vista por un usuario sin ser generada por el servidor. Por ejemplo, una página de un anuncio clasificado es servida unas diez veces en promedio desde la CDN luego de ser generada por el origen. Esto es, por cada visita, hay 0.1 pedidos al origen. Para el tracking, por cada visita, habrá necesariamente 1 pedido al origen, y que no puede ser salvado por la CDN. Es una cantidad de tráfico muy importante, y delegarlo a terceros también es un gran ahorro.

Esto ya se hace hoy en OLX: ya hay un tracking de visitas al sitio, pero es un tracking *plano*, un seguimiento del tráfico total en todas las páginas, de las cuales ninguna tiene variaciones. Google Analytics permite definir dimensiones personalizadas [8], de las cuales asignar un valor a cada visita. Usando una dimensión por cada experimento, se puede reutilizar el mismo evento de tracking ya existente y, además de no soportar el tráfico en el origen, tampoco hay que agregar un nuevo pedido desde el cliente. Entonces el tracking, al estar incluido en un costo que ya se está pagando, resulta gratuito.

3.2. Alternativas evaluadas

3.2.1. Selección de páginas completas o variación de fragmentos

Se hizo una primera prueba del concepto de delegar la decisión a la CDN sustituyendo la página en la que habría variaciones por un documento ESI que, en función del dado, incluyera la versión A o la versión B de la misma página, generada por separado. Se agregó una condición al inicio del proceso de generación de la página para que, si había un parámetro de entrada indicando una versión específica, se generara la página con las variaciones correspondientes a la versión y, si no había tal parámetro, se emitiera el documento ESI que hacía la selección. El bloque de la figura 11 es un pseudocódigo de esta rutina.

Figura 11. Primera implementación. La respuesta en la URL original es el documento ESI que incluye los dos contenidos alternativos pero selecciona sólo uno para la salida. La respuesta en las URLs con el parámetro adicional son las variaciones. El código en negro se ejecuta en el servidor, emitiendo a su salida el código en verde, que se ejecuta en la CDN.

```
if (NOT existe_parametro_entrada("version")) {

  print "
    <esi:choose>
      <esi:when test="$(dado) == 'A'">
        <esi:include src="/pagina?version=A"/>
      </esi:when>
      <esi:otherwise>
        <esi:include src="/pagina?version=B"/>
      </esi:otherwise>
    </esi:choose>
  "

} else {
```

```
  if (parametro_entrada("version") == "A") {

    print "
      <html>
        <body>Página versión A</body>
      </html>
    "

  } else {

    print "
      <html>
        <body>Página versión B</body>
      </html>
    "

  }
}
```

El usuario pedía la página indicando su dirección original (por ejemplo, `/pagina`). Como no había ningún parámetro de entrada, el origen respondía sólo el documento ESI con las instrucciones para la CDN. Esta evaluaba el dado y, en función del resultado, hacía un nuevo pedido al origen a la misma dirección pero con el parámetro de entrada que indicaba la versión. Esta vez, el origen respondía con una página para el cliente, en la versión solicitada, que la CDN incluía en su salida. El cliente recibía sólo la página que le correspondía y haciendo un solo pedido, por lo que el método era transparente para él, y la respuesta del origen en cada dirección (los parámetros son parte de ella) era constante y, por lo tanto, *reutilizable*.

Este método era funcionalmente correcto y el objetivo estaba cumplido, pero había espacio para mejorar en términos de performance. El contenido variado era generado sólo una vez, en la versión que le correspondía, pero el contenido común a las dos versiones era generado *dos veces*, una de ellas sin necesidad, sólo por no conservarlo. No era un malgasto menor pues, por definición de la técnica de experimentación A/B, el contenido variado suele ocupar una extensión significativamente menor que la del contenido común. Finalmente, toda la infraestructura que subyace a cualquier respuesta (framework, pila de ejecución, conexión HTTP, etc.), era puesta en marcha *tres veces*. Si x es el costo de generación de la página y v es la fracción de ese costo que corresponde al contenido variado, el costo total de generación de la página era

$$3x(v) + 3x(1-v) = 3x \qquad (3)$$

Se optó entonces por generar todo el contenido de una sola vez. La respuesta en la dirección original tiene ahora todo el contenido común y todas las versiones del contenido variado, condicionadas con el mismo código de decisión en función del valor del dado (figura 12).

Figura 12. Segunda implementación. La respuesta en la URL original es un documento ESI con el contenido original no variado y las variaciones envueltas en condicionales. Se destaca el código ESI que toma la decisión de qué contenido emitir a la salida.

```
<html>
  <body>

    <esi:choose>
      <esi:when test="$(dado) == 'A'">

        <esi:text>Página versión A</esi:text>

      </esi:when>
      <esi:otherwise>

        <esi:text>Página versión B</esi:text>

      </esi:otherwise>
    </esi:choose>

  </body>
</html>
```

La infraestructura es puesta en marcha una sola vez y el contenido común se genera también una sola vez. El costo de generar la página es ahora

$$3x(v) + 1x(1-v) = (2v + 1)x \qquad (4)$$

que en el peor caso, cuando la variación abarca la página entera, es igual a $3x$, pero que en la práctica, por la definición de la técnica, se aproxima a $1x$.

Fue necesario para esto modificar la interfaz de la librería. En su forma original, en la que cada generación de una respuesta de contenido correspondía a una sola versión, era posible condicionar la ejecución en el origen, utilizando la instrucción de selección del

lenguaje (la sentencia `if`). En su forma final, el código de todas las versiones debe ser ejecutado, por lo que se cambió a la forma de la figura 13.

Figura 13. Para emitir todas las variaciones en un mismo documento, hay que ejecutar en el servidor todas las ramas de la selección. La interfaz programática es entonces una función que recibe dos bloques de código, uno por tratamiento, uno por control. Se destaca la interfaz programática en el servidor.

```
<html>
  <body>

    variar("A", function() {

      Página versión A

    }, function() {

      Página versión B

    })

  </body>
</html>
```

La forma es visualmente similar para el programador, pero envolver las ramas condicionadas en funciones de retrollamada (o *callbacks*) permite delegar su ejecución a la función que las recibe y le permite a ella envolver la salida que emiten. La implementación de la función, entonces, puede ser la de la figura 14.

Figura 14. Pseudocódigo de la función de variación, que recibe la versión a la que debe condicionar las ramas, y una función de retrollamada para cada rama. Se destacan la información que recibe y cómo se usa.

```
function variar(version, fn_then, fn_else) {
    <esi:choose>

      <esi:when test="$(dado) == version">
        <esi:text>call(fn_then)</esi:text>
      </esi:when>

      <esi:otherwise>
        <esi:text>call(fn_else)</esi:text>
      </esi:otherwise>

    </esi:choose>
}
```

3.2.2. Evaluación en una pasada o en dos pasadas

Para poder seleccionar los bloques de código que serán emitidos a la salida, es necesario saber cuál de las versiones que un experimento propone será presentada. Esto se calcula encontrando el valor resultante de la evaluación del dado en la distribución de probabilidades definida para el experimento. Siendo necesaria para la selección de los bloques de contenido, tal evaluación debe ejecutarse necesariamente antes de la primera instancia de selección.

Inicialmente, el código para la evaluación del dado y la selección de la versión a presentar estaban al inicio del documento y ocurrían al principio de la ejecución. El documento de salida consistía en la evaluación del dado, la selección de la versión de cada experimento activo y finalmente la emisión del contenido con los bloques condicionados (figura 15).

Figura 15. Primera implementación. Lectura y almacenamiento del dado, selección de las variaciones, y condicionamiento de la emisión del contenido. Se destaca el código ESI completo.

```
// lectura y almacenamiento del dado
<esi:assign name="dado" value="$tirarDado()"/>

// selección de las variaciones
<esi:assign name="test1" value="$elegirVariacion($(dado), 50)"/>

// condicionamiento de la emisión del contenido
<html>
  <body>

    <esi:choose>
      <esi:when test="$(test1) == 'TRATAMIENTO'">

        <h1>Avisos Clasificados</h1>

      </esi:when>
      <esi:otherwise>

        <h1>Anuncios Clasificados</h1>

      </esi:otherwise>
    </esi:choose>

  </body>
</html>
```

Funcionalmente, esta implementación era correcta, pero todas las páginas tenían el código de selección de versión de todos los experimentos activos, aún cuando no hubiera variaciones de todos los experimentos en todas las páginas. Al crecer la cantidad de experimentos y la cantidad de páginas, el volumen de código y de ejecución crecían cuadráticamente cuándo sólo estaban justificados linealmente (figura 16).

Figura 16. Pseudocódigo de dos páginas, una (página principal) en la que se varía por un experimento (testAvisoAnuncio), otra (página de publicación) en la que se varía por otro experimento (testBotónLink). La selección de variaciones se evalúa para todos los experimentos activos, independientemente de si hay condicionamientos a ellos en la respuesta. En ambas páginas, el bloque inicial es igual. Se destacan las asignaciones y las lecturas de las variaciones elegidas.

```
// Página principal en la que se varía sólo por "testAvisoAnuncio"

<esi:assign name="dado" value="$tirarDado"/>
<esi:assign name="testAvisoAnuncio"
            value="$elegirVariacion($(dado), 50)"/>
<esi:assign name="testBotonLink"
            value="$elegirVariacion($(dado), 25)"/>

<html>
  <body>
    <h1>Página principal</h1>
    <a href="/publicar">Publique su

      <esi:choose>
        <esi:when test="$(testAvisoAnuncio) == 'TRATAMIENTO'">
          aviso clasificado
        </esi:when>
        <esi:otherwise>
          anuncio clasificado
        </esi:otherwise>
      </esi:choose>

    </a>
  </body>
</html>

// Página de publicación en la que se varía sólo por "testBotonLink"

<esi:assign name="dado" value="$tirarDado"/>
<esi:assign name="testAvisoAnuncio"
            value="$elegirVariacion($(dado), 50)"/>
<esi:assign name="testBotonLink"
            value="$elegirVariacion($(dado), 25)"/>
```

```
<html>
  <body>
    <h1>Publicar un clasificado</h1>
    <form>
      ...(campos del formulario)...

      <esi:choose>
        <esi:when test="$(testBotonLink) == 'TRATAMIENTO'">
          <input type="submit" value="Publicar"/>
        </esi:when>
        <esi:otherwise>
          <a onclick="this.form.submit()">Publicar</a>
        </esi:otherwise>
      </esi:choose>

    </form>
  </body>
</html>
```

Se necesitaba evaluar el dado y elegir la versión a presentar sólo para aquellos experimentos que tuvieran variaciones en la página.

Se cambió entonces a una implementación por demanda: no se evaluaría nada al inicio y, ante la primera variación, se emitiría el código de evaluación y la variación inmediatamente después. Así, seguiría cumpliéndose la condición de evaluar antes de seleccionar, pero sólo se evaluaría en el caso que efectivamente hubiera variaciones en la página (figura 17).

Figura 17. Pseudocódigo de dos páginas, una (página principal) en la que se varía por un experimento (testAvisoAnuncio), otra (página de publicación) en la que se varía por otro experimento (testBotónLink). La selección de cada variación ocurre por demanda inmediatamente antes del primer condicionamiento. Se destacan las asignaciones y las lecturas de las variaciones elegidas.

```
// Página principal en la que se varía sólo por "testAvisoAnuncio"

<esi:assign name="dado" value="$tirarDado"/>
<html>
  <body>
    <h1>Página principal</h1>
    <a href="/publicar">Publique su

      // selección de variación de testAvisoAnuncio
      <esi:assign name="testAvisoAnuncio"
                  value="$elegirVariacion($(dado), 50)"/>
```

```
      // condicionamiento del contenido según testAvisoAnuncio
      <esi:choose>
        <esi:when test="$(testAvisoAnuncio) == 'TRATAMIENTO'">
          aviso clasificado
        </esi:when>
        <esi:otherwise>
          anuncio clasificado
        </esi:otherwise>
      </esi:choose>

    </a>
  </body>
</html>

// Página de publicación en la que se varía sólo por "testBotonLink"

<esi:assign name="dado" value="$tirarDado"/>
<html>
  <body>
    <h1>Publicar un clasificado</h1>
    <form>
      ...(campos del formulario)...

      // selección de variación de testBotonLink
      <esi:assign name="testBotonLink"
                  value="$elegirVariacion($(dado), 25)"/>


      // condicionamiento del contenido según testBotonLink
      <esi:choose>
        <esi:when test="$(testBotonLink) == 'TRATAMIENTO'">
          <input type="submit" value="Publicar"/>
        </esi:when>
        <esi:otherwise>
          <a onclick="this.form.submit()">Publicar</a>
        </esi:otherwise>
      </esi:choose>

    </form>
  </body>
</html>
```

Esta implementación era más eficiente, ya que no había evaluaciones innecesarias, pero lamentablemente era funcionalmente incorrecta: cuando la primera variación estaba sujeta en ESI a una condición externa (por un uso de ESI con otro propósito, ajeno al módulo de experimentación A/B), la evaluación no ocurría nunca. Y si había una segunda variación

fuera de la condición ajena, no habría información para ejecutar la selección y ocurriría un fallo (figura 18).

Figura 18. Pseudocódigo de una página con dos bloques condicionados, con la selección de variación inmediatamente antes del primer bloque, y una condición externa afectando todo el primer bloque. Si la condición externa no se cumple, en el segundo bloque no habrá información suficiente para decidir. Se destacan la condición externa que puede impedir la selección de la variación, la selección sujeta a la condición y la lectura de la selección fuera de aquella.

```
<esi:assign name="dado" value="$tirarDado"/>
<html>
  <body>

    <esi:choose>
      <esi:when test="$(GEO{country}) == 'ARGENTINA'">
        <h1>

          // selección de variación
          <esi:assign name="testAvisoAnuncio"
                      value="$elegirVariacion($(dado), 50)"/>

          // primer bloque condicionado
          <esi:choose>
            <esi:when test="$(testAvisoAnuncio) == 'TRATAMIENTO'">
              Avisos clasificados
            </esi:when>
            <esi:otherwise>
              Anuncios clasificados
            </esi:otherwise>
          </esi:choose>

        </h1>
      </esi:when>
    </esi:choose>

    </h1>
    <a href="/publicar">Publique su

      // segundo bloque condicionado
      <esi:choose>
        <esi:when test="$(testAvisoAnuncio) == 'TRATAMIENTO'">
          aviso clasificado
        </esi:when>
        <esi:otherwise>
          anuncio clasificado
        </esi:otherwise>
      </esi:choose>

    </a>
```

```
  </body>
</html>
```

Era necesario entonces hacer que la evaluación fuera *incondicional*, y la única forma de proteger el bloque de evaluación de cualquier condición externa era quitarlo del bloque de contenido. Como las variaciones están dentro del bloque de contenido y la evaluación debe ocurrir antes, había que volver a ponerlas al inicio y encontrar otra manera de saber cuándo evaluar cada experimento.

Una primera aproximación fue la implementación de una selección *declarativa* del conjunto de páginas en las que cada experimento sería evaluado: en la definición de cada experimento, además de la definición de las versiones y su reparto del espacio de probabilidad, se indicaría el conjunto de páginas en las cuales el experimento estaría activo. Así, el código de evaluación sólo se emitiría en las páginas que pertenecieran al conjunto (figura 19).

Figura 19. Definición declarativa de dos experimentos, incluyendo la lista de páginas que cada uno afecta, y pseudocódigo de tales páginas, cada uno con la evaluación de sólo aquellos experimentos a los cuales tiene código condicionado, nuevamente al inicio. Se destacan las instrucciones de selección de variación de sólo los experimentos activos en cada página.

```
// Definición declarativa de los tests

<testsuite>
  <test nombre="testAvisoAnuncio" porcentajeTratamiento="50">
    <pagina>/</pagina>
  </test>
  <test nombre="testBotonLink" porcentajeTratamiento="25">
    <pagina>/publicar</pagina>
  </test>
</testsuite>

// Página principal en la que se varía sólo por "testAvisoAnuncio"

<esi:assign name="dado" value="$tirarDado"/>
<esi:assign name="testAvisoAnuncio"
            value="$elegirVariacion($(dado), 50)"/>
<html>
  <body>
    <h1>Página principal</h1>
    <a href="/publicar">Publique su
```

```
      <esi:choose>
        <esi:when test="$(testAvisoAnuncio) == 'TRATAMIENTO'">
          aviso clasificado
        </esi:when>
        <esi:otherwise>
          anuncio clasificado
        </esi:otherwise>
      </esi:choose>

    </a>
  </body>
</html>

// Página de publicación en la que se varía sólo por "testBotonLink"

<esi:assign name="dado" value="$tirarDado"/>
<esi:assign name="testBotonLink"
            value="$elegirVariacion($(dado), 25)"/>
<html>
  <body>
    <h1>Publicar un clasificado</h1>
    <form>
      ...(campos del formulario)...

      <esi:choose>
        <esi:when test="$(testBotonLink) == 'TRATAMIENTO'">
          <input type="submit" value="Publicar"/>
        </esi:when>
        <esi:otherwise>
          <a onclick="this.form.submit()">Publicar</a>
        </esi:otherwise>
      </esi:choose>

    </form>
  </body>
</html>
```

Esto redujo la sobreevaluación cuadrática de la primera implementación, pero introdujo nuevos problemas. El primero fue el acoplamiento entre el conjunto declarado y la presencia de las variaciones en el contenido: cada vez que se agregaba una variación en una página que no tenía o que se quitaba una variación en una página en la que no quedaban más, el conjunto cambiaba y el desarrollador debía ajustar el conjunto declarado, o habría una inconsistencia. Los falsos positivos no eran el problema: si una página estaba en el conjunto declarado y luego no tenía variaciones, se haría una evaluación innecesaria, reduciendo la ineficiencia pero no fallando. El problema eran los falsos negativos: si el conjunto declaraba

que una página no tendría variaciones y luego las tenía, estas serían ejecutadas sin la evaluación del dado y fallarían. Así, la consistencia entre el conjunto declarado y el conjunto efectivo no sólo requería trabajo adicional sino que imponía un riesgo de errores en tiempo de ejecución sujeto a la memoria humana y su imperfección.

El segundo problema es que hay casos en los que, aún sin errores humanos, tal consistencia es inalcanzable. Tal como en el *Problema de la parada* [10] es imposible saber algorítmicamente si un programa con una determinada entrada terminará y, por extensión, si se ejecutará o no una instrucción arbitraria de las que lo componen, cuando la presencia de una o más variaciones está condicionada por uno o más valores de la entrada, resulta imposible saber *a priori* (esto es, antes de evaluar el código) si tales variaciones estarán presentes en la salida. Y si todas las variaciones de un mismo experimento en una misma página están condicionadas, tampoco se puede saber *a priori* si el experimento mismo debe ser evaluado o no en esa página y, entonces, si debe o no formar parte del conjunto declarado (figura 20).

Figura 20. Pseudocódigo de una página con una variación condicionada a un valor de entrada. Es imposible saber *a priori* si la variación estará presente en la salida, para emitir o no, antes, la selección de la variación. Se destacan la selección incondicional de la variación y la condición en el servidor que puede impedir la lectura de la variación en la salida.

```
<esi:assign name="dado" value="$tirarDado"/>

// selección de la variación, que puede estar de más
<esi:assign name="test1"
            value="$elegirVariacion($(dado), 25)"/>

<html>
  <body>

    <?php if (preg_match('google.com', $_SERVER['HTTP_REFERER'])) { ?>

      <h1 id="titulo">
        <esi:choose>
          <esi:when test="$(test1) == 'TRATAMIENTO'">
            Avisos Clasificados
          </esi:when>
          <esi:otherwise>
            Anuncios Clasificados
          </esi:otherwise>
        </esi:choose>
      </h1>

    <?php } ?>
```

```
  </body>
</html>
```

El origen genera la página en una sola pasada: a medida que genera el contenido, va emitiéndolo a la salida. No hay forma de generarlo y, al final del proceso, agregar algo al inicio. Si el conjunto de experimentos que deben ser evaluados sólo se conoce después de generar (y emitir) el contenido, el código de evaluación también sólo puede emitirse después. Fue necesario entonces encontrar una forma de que el código de evaluación estuviera ubicado al final del documento pero fuera ejecutado al principio.

El objetivo final del experimento es saber cuál versión, cuál conjunto de variaciones, produce la mejor tasa de conversión, es decir, el mejor cociente entre el número de conversiones y el número de visitas. Ya se registraron hace un largo tiempo las visitas y las conversiones con el servicio de Google Analytics. Para asignar cada visita o conversión a una versión particular de un experimento, bastará agregar al registro una etiqueta indicando la asignación. Luego, al observar los resultados, se segmentará el tráfico por el valor de la etiqueta y se podrán comparar las tasas de conversión.

La rutina de tracking de Google Analytics está implementada en Javascript, y es incluida en las páginas web como un script más. Para que la rutina pueda etiquetar los registros, debe contar con la lista de experimentos activos y la versión asignada a cada uno. Sólo deben contarse los casos en los que el usuario haya sido efectivamente expuesto al experimento, aquellos casos en los que se le haya entregado contenido que varíe entre una versión y otra y que pueda influir positiva o negativamente en su probabilidad de conversión. Una conversión de un usuario que no fue expuesto a ninguna variación no debe ser contada a favor de ninguna versión. Hay entonces que exportar la lista de experimentos activos y sus asignaciones a javascript *después* de la evaluación de los experimentos para conocer el conjunto efectivo, pero *antes* de la rutina de tracking para que cuente con la información al momento de su ejecución.

En la primera versión, aquella en la que el código de evaluación de los experimentos estaba ubicado al inicio del documento que recibía la CDN, se emitía a la salida inmediatamente después un bloque de código javascript que guardaba en variables la lista de experimentos y las asignaciones, dejándolas disponibles para que la rutina de tracking los tomara en cualquier momento. Lamentablemente sufría del mismo problema mencionado antes: se

etiquetaban registros con asignaciones de versiones incluso cuando el usuario aún no hubiera sido expuesto a ellas.

La tasa de abandono de un sitio web es la fracción de todas las personas que pidieron una página del sitio que abandonan el pedido antes de que la página se cargue. Esas visitas abandonadas tuvieron un costo publicitario para que ocurrieran y un costo operativo cuando se hicieron efectivas, y el usuario no fue a cambio expuesto al producto, por lo que representan una pérdida económica y es interés del negocio reducirlas al mínimo.

La evaluación subjetiva del usuario de la velocidad de descarga de una página no depende tanto del tiempo de transferencia de la página completa como del tiempo que transcurre entre el pedido de la página y el comienzo del rendering. Para mejorar la experiencia del usuario, entonces, los navegadores comienzan el rendering tan pronto tienen información suficiente para hacerlo, sin esperar a recibir el documento HTML completo. Como la tasa de abandono está inversamente correlacionada con la sensación de velocidad [11], en la construcción de una página se busca optimizar el proceso de carga de manera que no sólo la transferencia total sea mínima, sino que también el rendering pueda comenzar lo más temprano posible en ese período de transferencia. Por ejemplo, las referencias a hojas de estilo (necesarias para el rendering) suelen estar ubicadas al principio del documento HTML, para que sean encontradas y resueltas aún si el documento todavía está siendo transferido, mientras que los scripts (no necesarios) suelen estar ubicados al final [9].

Ya están en uso en las páginas de OLX la mayoría de estas prácticas de optimización y, en particular, la de ubicar todos los scripts al final del cuerpo de la página. Tal es así que el desarrollador cuenta con la garantía de que cualquier variación de contenido visible para el usuario será emitida en la salida *antes* de ser emitida la rutina de tracking. Esto facilitó una segunda implementación, en la cual se esperaba a ejecutar la generación de todo el contenido visible, terminada la cual se conocía la lista de experimentos activos, para inmediatamente después exportar la lista a javascript como primer elemento de la lista de scripts enviados al final del documento (figura 21).

Figura 21. Pseudocódigo de una página, con una variación de contenido visible, luego la exportación a javascript de la lista de experimentos activos, *después* la rutina de tracking, y finalmente el resto de los scripts.

```
<esi:assign name="dado" value="$tirarDado"/>
<esi:assign name="test1"
            value="$elegirVariacion($(dado), 25)"/>
<html>
```

```
  <body>

    // variación de contenido visible
    <h1 id="titulo">
      <esi:choose>
        <esi:when test="$(test1) == 'TRATAMIENTO'">
          Avisos Clasificados
        </esi:when>
        <esi:otherwise>
          Anuncios Clasificados
        </esi:otherwise>
      </esi:choose>
    </h1>

    // exportación a javascript del resultado de la selección
    <script>
      var tests = {};
      tests{'test1'} = '<esi:vars name="test1"/>';
    </script>

    // rutina de tracking
    <script src="googleanalytics.js"></script>
    <script>
      var ga = iniciarGoogleAnalytics();
      for (var nombre in tests) {
        ga.agregarDimension(nombre, tests[nombre]);
      }
      ga.trackPageView();
    </script>

    // otros scripts
    <script src="otro1.js"></script>
    <script src="otro2.js"></script>

  </body>
</html>
```

Esta implementación resultó satisfactoria funcionalmente, pero no en términos de calidad, pues imponía una limitación arbitraria al desarrollador y lo obligaba a conocer la ubicación física de esa limitación, violando el encapsulamiento. El desarrollador podía indicar variaciones por experimento en cualquier lugar del documento que, en la salida, quedase ubicado *antes* del primer script del conjunto de scripts del final del documento. No podía, sin embargo, variar el contenido de los scripts, pues las variaciones serían detectadas tardíamente y podrían no ser registradas. Incluso, el framework usado para la generación de las páginas tiene la capacidad de retener los scripts que se generan como parte del

contenido y no emitirlos a la salida sino al final, de manera automática y transparente para el desarrollador, por lo que este, aún teniendo la precaución de no sobrepasar el límite deliberadamente, podría sobrepasarlo accidentalmente por acción de la plataforma.

Por otro lado, la práctica ya instalada en el sitio web de OLX de enviar los scripts al final es resultado de un proceso de optimización que no ocurrió al inicio del desarrollo sino bastante tiempo más tarde, con el producto ya consolidado y funcionando en el mercado. Un nuevo producto de OLX, o un producto ajeno en el caso de distribuir esta pieza de software a terceros, podría no cumplir con esta práctica. Si se la impone como condición, se reduce significativamente el espacio de utilidad.

Era deseable, entonces, poder emitir la lista de experimentos y sus asignaciones no *cerca* del final del documento sino *estrictamente* al final, permitiendo al desarrollador desconocer la implementación y variar libremente cualquier pieza de contenido. Y a la vez, para no asumir una determinada ubicación de la rutina de tracking y permitir a otras piezas de software ubicarla en cualquier lugar del documento, era deseable tener disponible la exportación no cerca del principio de la ejecución, sino estrictamente al principio. Había que encontrar una manera de que la generación de la información de tracking quede ubicada al final del documento pero que pueda ser ejecutada al principio.

La solución se conoce como inversión de control [12]. La salida del origen tiene una parte principal, el contenido de la página, que se emite primero, y dos partes secundarias, la evaluación de los experimentos y la exportación del resultado al cliente, que sólo pueden ser emitidas al final, pero que deben ejecutarse en momentos precisos dentro de la ejecución de la parte principal. El contenido de la página es emitido en forma de una función, seguido de un llamado a esa misma función, pasándole el control principal y dos funciones argumento para inyectar las dependencias (figura 22).

Figura 22. Pseudocódigo solución. Una función principal con todo el contenido que debe ser emitido al usuario, menos los bloques de evaluación y exportación, expresados como llamados a funciones de retrollamada que inyectan el código restante. De esta manera, pueden ser emitidos a la salida del servidor *después* del contenido, pero emitidos a la salida de la CDN *antes*. Se destacan los argumentos y su uso como llamados a función.

```
// contenido emitido antes y ejecutado después
function main(fn_evaluar, fn_exportar) {

      fn_evaluar()

      <contenido>
      if (versiones(test1) == 'A') {
```

```
            <contenido de la versión A>
      }
      <contenido>

      fn_exportar()

      <otros scripts>

}

// contenido emitido después y ejecutado antes
main(function() {
      dado = tirar_dado()
      versiones = seleccionar_versiones(dado)
}, function() {
      print to_javascript(versiones)
})
```

Lamentablemente, hay una limitación en el lenguaje ESI que impidió la implementación directa de esta solución: las funciones en ESI no pueden emitir salida. Están concebidas para procesar información y retornarla. Pueden contener instrucciones que emitan salida, pero la salida emitida es descartada.

Fue necesario entonces hacer parte del trabajo que haría el compilador si el lenguaje soportara las instrucciones de la solución, para bajar el grado de abstracción hasta un nivel soportado. Esto es: pasar todo el código al ámbito global, pasar la información entre bloques con variables globales bien controladas, y dirigir el flujo de ejecución con saltos condicionados. Aún así, los saltos *per se* no están soportados, pero hay instrucciones de nivel superior, como la iteración y la selección, con las cuales implementarlos.

La implementación final es severamente cuestionable en términos de claridad del concepto y representación de la solución, pero no debe olvidarse que es código derivado, generado automáticamente a partir del uso de una interfaz programática, y que debe ser juzgado como el código en lenguaje de ensamble que genera un compilador (figura 23).

Figura 23. Pseudocódigo de la implementación final de la solución, sujeta a las limitaciones del lenguaje. Se destacan las instrucciones que definen el flujo de ejecución.

```
run_user_code = false
do {
      if (run_user_code) {

            // contenido emitido antes y ejecutado después
```

```
            <contenido>
            if (versiones(test1) == 'A') {
                  <contenido de la versión A>
            }
            <contenido>
            print to_javascript(versiones)
            <otros scripts>

      } else {

            // contenido emitido después y ejecutado antes
            dado = tirar_dado()
            versiones = seleccionar_versiones(dado)

      }
} while run_user_code = !run_user_code
```

3.2.3. Tracking del dado o de la variación

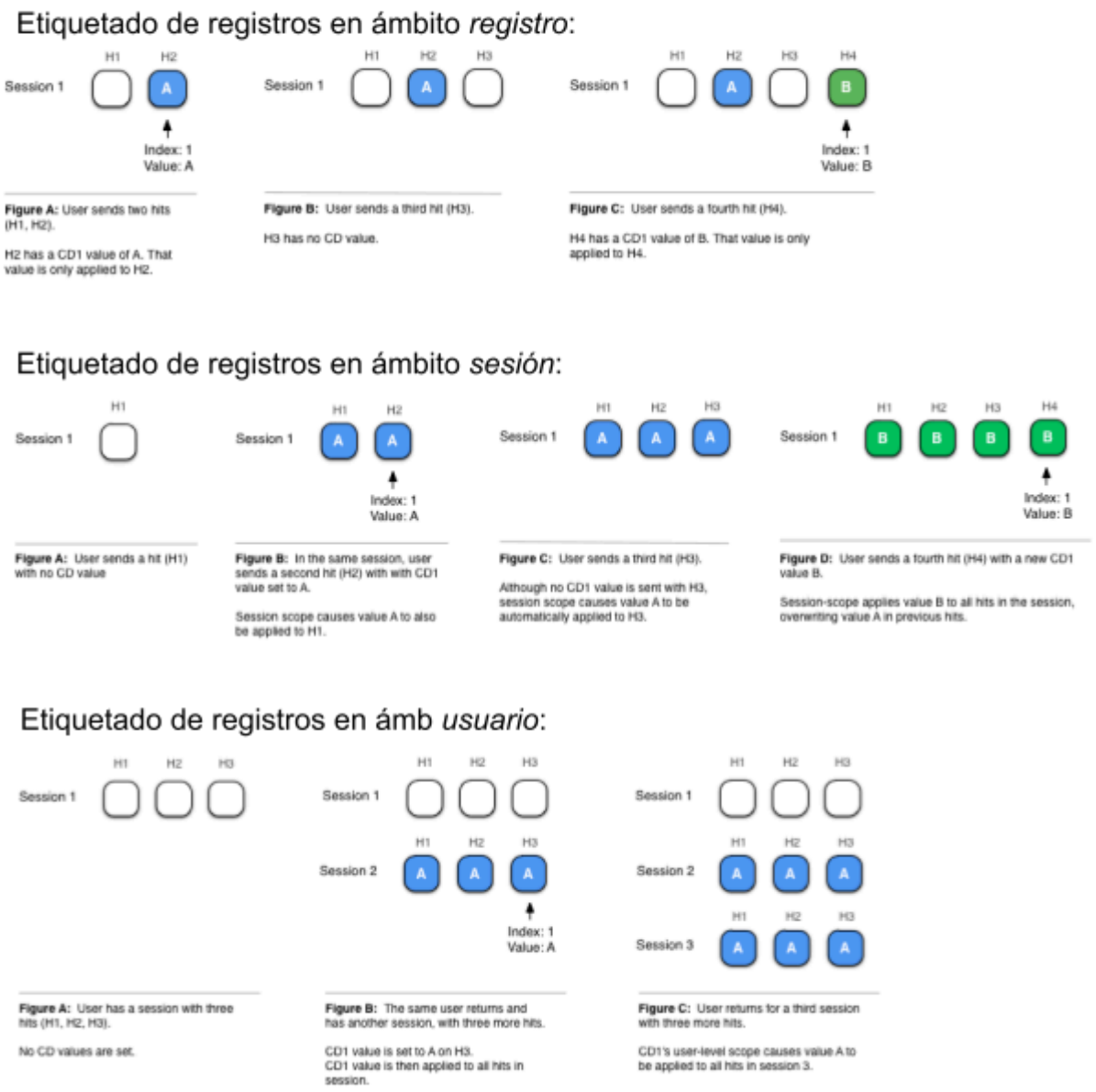
Google Analytics permite adjuntar a cada registro de una visita o un evento hasta 5 pares clave+valor. El servicio de Google ya etiqueta cada registro con datos de propósito general como la ubicación geográfica o el tipo de dispositivo del usuario. Con estas etiquetas particulares, es posible asociar los registros a datos específicos del negocio, como una categoría a la que el usuario pertenece, o si está registrado y ha iniciado sesión.

Las etiquetas pueden ser aplicadas en tres ámbitos o scopes: registro, sesión y usuario. Una etiqueta aplicada en el ámbito registro sólo queda asociada al registro en el cual fue aplicada. Una etiqueta aplicada en el ámbito sesión queda asociada a todos los registros de la sesión de la que forma parte el registro en la que fue aplicada. Una etiqueta en el ámbito usuario queda asociada a todos los registros del usuario a partir de la sesión de la que forma parte el registro en la que fue aplicada.

La figura 24 detalla los tres ámbitos con la explicación del autor de la herramienta. Cada cuadro representa una página pedida y cada fila representa una visita a una o más páginas. En el ámbito *registro*, una visita o sesión puede tener más de una etiqueta, porque cada etiqueta abarca sólo el registro de una página. En el ámbito *sesión*, no bien se etiqueta un registro, toda la sesión queda etiquetada, y si en un registro posterior aparece una nueva etiqueta, la primera resulta sobreescrita. En el ámbito *usuario*, no bien se etiqueta un

registro, toda la sesión queda etiquetada, junto con todas las sesiones posteriores del usuario.

Figura 24. Ámbitos de etiquetado de registros en Google Analytics.

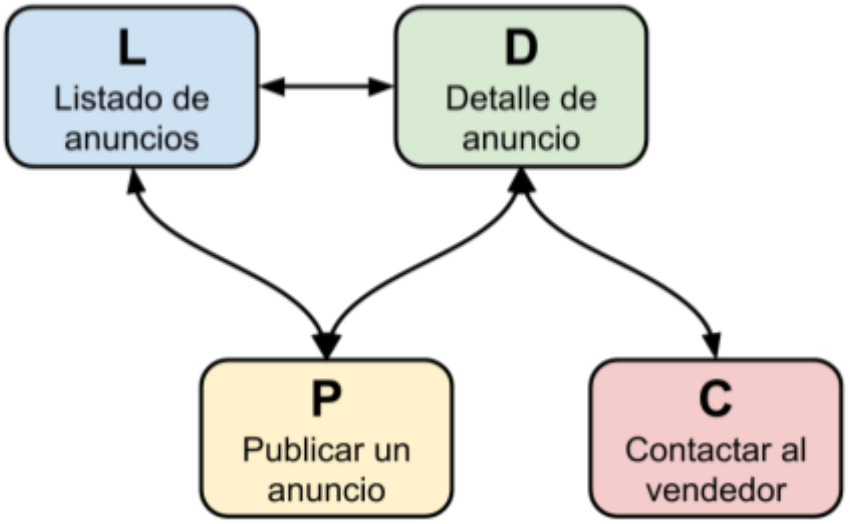


La primera implementación consistió en tirar un dado y guardarlo en una etiqueta con ámbito usuario, etiquetando de esta manera todos los registros de acciones del usuario con el valor del dado. El dado tenía un valor entre 00 y 99 y el porcentaje del tráfico que se asignaba a cada variación se asociaba a un rango proporcional de sus valores. Luego, al observar el registro, se podía saber a qué variación correspondía cada meta por el valor de

dado que tenía en su etiqueta. Como se guardaba el valor del dado y no la variación elegida, el valor almacenado servía para los tests subsiguientes: no hacía falta tirar un nuevo dado cada vez.

La figura 25 muestra el esquema de un autómata finito que representa un sitio básico de avisos clasificados y la circulación posible por sus páginas, que será usado como ejemplo en la explicación subsiguiente.

Figura 25. Autómata finito que representa un conjunto de páginas que forman un sitio básico de avisos clasificados y la posible circulación entre ellas.



La siguiente tabla muestra 4 ejemplos de visitas al sitio, con un experimento activo en el cual la conversión es visitar la página de publicación. Sólo hay conversión en una de las cuatro visitas. El tratamiento recibió el 50% del tráfico y el control el 50% restante. El resultado con esta muestra es que la versión de control tuvo un 0% de conversión y la versión de tratamiento tuvo un 50% de conversión. La muestra tiene que ser más grande para que estos resultados sean confiables, pero la definición del resultado proviene de la misma comparación de cocientes.

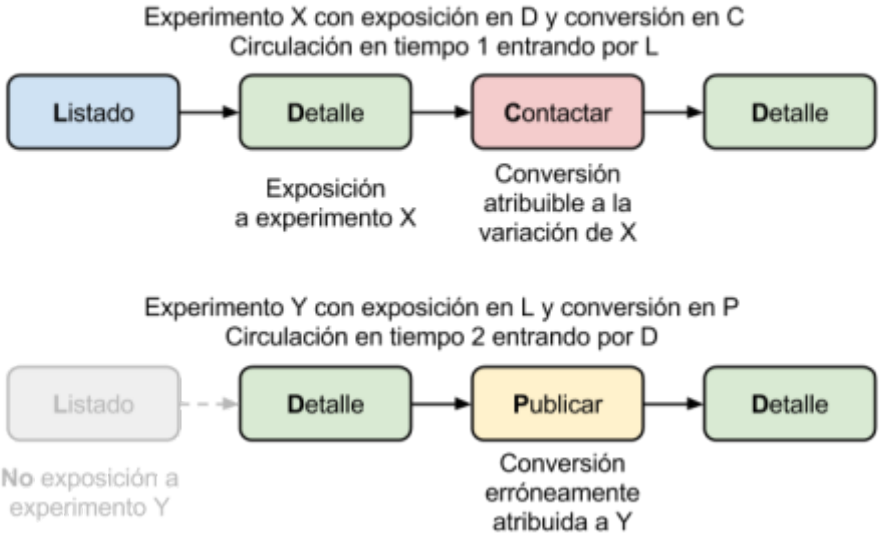
Tabla 2. Ejemplos de circulación entre páginas, con conversiones y variaciones.

Circulación	**Conversión**	**Dado**	**Versión**
LDLDPL	Sí	52	Tratamiento
LDCD		17	Control

LDLDLD		31	Control
DLD		89	Tratamiento

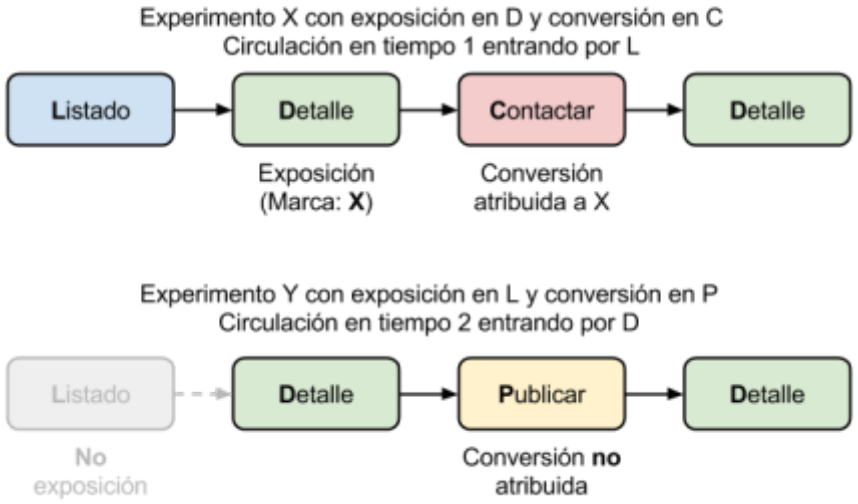
Esto funcionó bien mientras la variación y la meta estaban en la misma página. Cuando la meta estaba en una página distinta, a la que se podía llegar sin pasar por aquella en la que estaba la variación, la meta podía ocurrir aún sin que el usuario fuera expuesto. Es fácil excluir estos casos no contando las metas que no tengan un valor de dado asociado, pero esos casos son sólo aquellos en los que el usuario no fue expuesto a ningún test en su historia: si el usuario ya había participado en otro test, ya tenía el dado tirado y asociado, y la nueva meta sería etiquetada con una variación aún cuando ninguna exposición haya ocurrido (figura 26).

Figura 26. Un usuario participa en un experimento X y le queda el dado tirado. Luego convierte una meta del experimento Y sin pasar por la exposición. La conversión es erróneamente atribuida a la variación de Y.



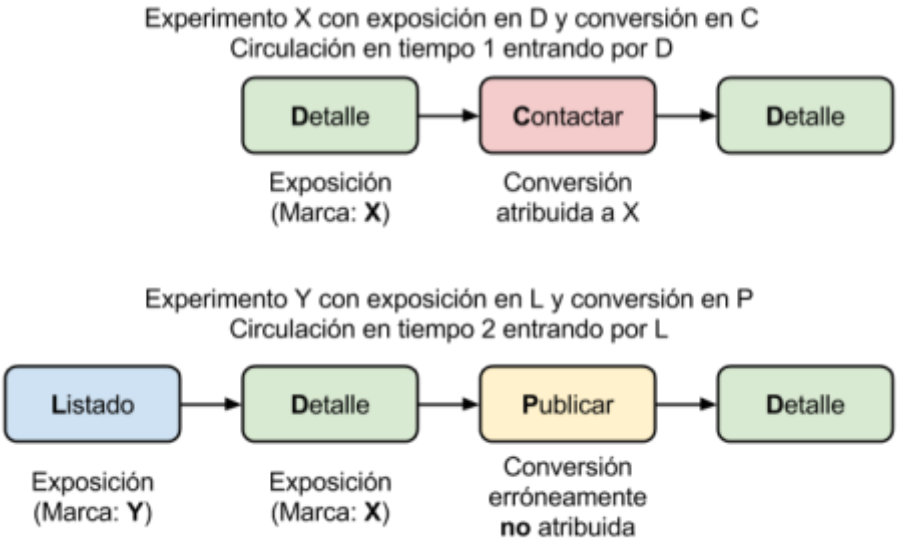
Agregamos a los registros una marca de exposición. Cuando el usuario pasaba por la página en la que estaba la variación, se agregaba la etiqueta passed_abtest=nombre_del_test. La etiqueta tenía el ámbito sesión, por lo que la sesión completa resultaba marcada. Al contar los registros de meta con cada variación, sólo se incluirían aquellos que tuviesen la etiqueta indicando que el usuario había sido efectivamente expuesto a la variación (figura 27).

Figura 27. La misma situación que en la figura 26, pero al no estar la marca de exposición en el registro de la conversión, la conversión no resulta atribuida al experimento.



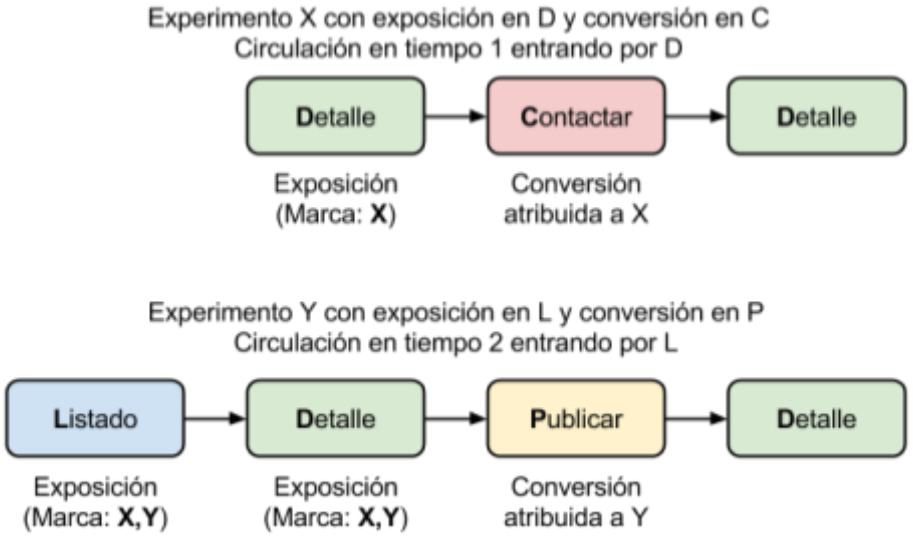
El siguiente obstáculo fue que no se podían hacer experimentos simultáneos: los registros de meta no se perdían pero, cuando el usuario era expuesto a más de un experimento, la marca de exposición al segundo experimento en la sesión sobreescribía la marca de exposición al primer experimento, invalidando los resultados de éste (figura 28).

Figura 28. Un usuario es expuesto a una variación del experimento X y su conversión en la siguiente página es correctamente atribuida. Luego es expuesto a una variación del experimento Y por la cual ocurre otra conversión, pero la exposición intermedia a otro experimento sobre escribe la marca.



Redefinimos la etiqueta ya no como un nombre sino como una lista de nombres, en la que cada exposición agregaba un elemento. De esa manera, el usuario podía ser expuesto a múltiples experimentos y no se perdería información (figura 29).

Figura 29. La misma situación que en la figura 28 pero, almacenando toda la lista de exposiciones en la marca, ambas conversiones son correctamente atribuidas.

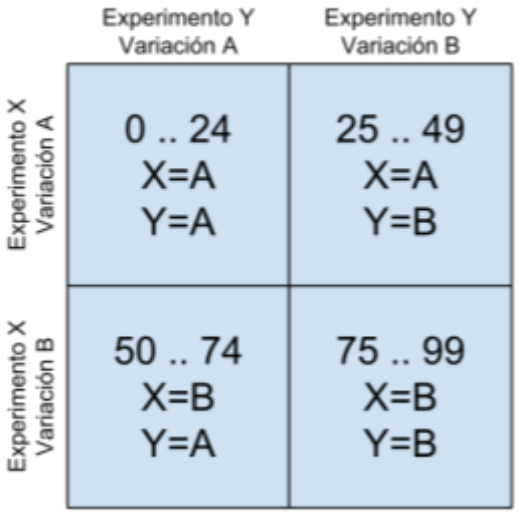


La lista funcionaba satisfactoriamente, aunque su implementación no era simple. No se podía simplemente agregar un elemento en cada exposición, porque crecería indefinidamente: había que agregar el elemento sólo si no estaba ya. Pero entonces la nueva estructura exigía una lógica de la misma complejidad que aquella que inicialmente queríamos evitar tirando un solo dado.

Por otro lado, el mecanismo de definir manualmente los rangos imponía una dificultad al operador, porque repartir proporcionalmente las probabilidades no era suficiente para una operacion segura: dos experimentos cualesquiera, sin importar si son simultáneos o sucesivos, pueden estar correlacionados y es necesario protegerse de esa correlación. Si se hacen dos experimentos, en los dos se asigna el rango 0-49 a la variación de tratamiento, y las variaciones son contradictorias, o el público prefiere una o la otra pero no las dos juntas, ganará el tratamiento en ambos experimentos, y se concluirá erróneamente que ambas variaciones son malas, cuando sólo lo son juntas, o para el mismo público. Para protegerse de eso, hay que repartir el espacio de probabilidades de manera que cada variación de un experimento tenga público de todas las variaciones del otro experimento en las mismas proporciones. Si la distribución fuera aleatoria, esto se conseguiría automáticamente. Si la distribución es manual, hay que considerar cada experimento como

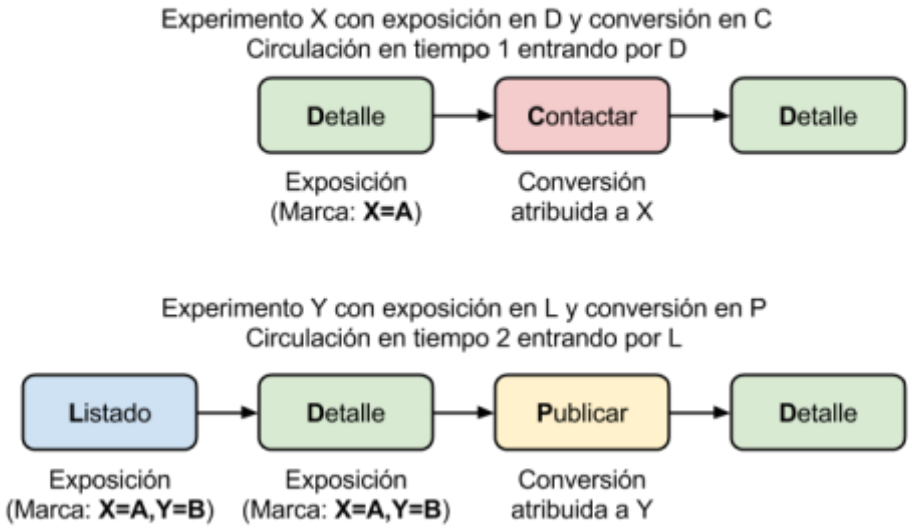
una dimensión y construir la matriz correspondiente. Por ejemplo, para dos experimentos con 50% de probabilidad de tratamiento, si en uno se asigna el rango 0-49 al tratamiento y el rango 50-99 al control, en el otro se podría asignar los rangos 0-24 y 50-74 al tratamiento y 25-49 y 75-99 al control, como muestra la figura 30.

Figura 30. Distribución del espacio de valores del dado entre las variaciones de diferentes experimentos para eludir la correlación.



Decidimos entonces tirar un dado por experimento. En el momento de la exposición a la variación de un experimento, se tiraba un dado particular del experimento, se elegía la variación y se almacenaba en una etiqueta con ámbito usuario. La etiqueta, nuevamente, era una lista aunque, esta vez, ya no de nombres sino de pares nombre_de_test + variación_elegida (figura 31).

Figura 31. Al tirar un dado por experimento, fue necesario almacenar, además de la ocurrencia de la exposición, la identificación de la variación a la cual el usuario había sido expuesto. El resultado es una lista de pares experimento:variación.



También esta implementación tuvo sus inconvenientes: la lista crecía indefinidamente, aún sin repeticiones, con la acumulación de ejecuciones de experimentos. No había forma de ir eliminando de ella los tests concluidos sin publicar la lista completa de tests concluidos en todas las páginas para que los clientes pudieran quitar de sus listas todos aquellos que aún estuvieran presentes.

El camino de agregar una marca, agregar varias marcas, guardar las variaciones o tirar un dado por cada experimento, no llevó a buen puerto, por lo que volvimos al inicio. El dado debía ser uno solo y constante, para no acumular información del lado del cliente que luego hubiera que mantener. La solución consistió en variar entonces, en lugar del valor del dado del lado del cliente, *la interpretación* de su valor del lado del servidor.

La primera variación consistió en tirar un conjunto de dados en lugar de sólo uno, un conjunto finito y constante de dados que se tiran todos al inicio, como muestra la figura 32.

Figura 32. Almacenamiento de un conjunto de 64 dados de dos dígitos como un string de 128 caracteres con números del 0 al 9. Se destacan la lectura y generación del conjunto.

```
<esi:choose>
  <esi:when test="$(COOKIE{dado}) matches /^([0-9][0-9]){64}$/">
    <esi:assign name="dado" value="$(COOKIE{dado})"/>
  </esi:when>
  <esi:otherwise>
    <esi:assign name="dado" value="'"/>
```

```
    <esi:foreach sequence="[1..64]" variable="i">
      <esi:assign name="dado" value="$(dado) + str(rand(100))"/>
    </esi:foreach>
  </esi:otherwise>
</esi:choose>
```

A cada experimento se le asigna uno de los *n* dados: al momento de evaluar el dado y decidir la variación de un experimento, sólo se consulta el valor del dado asignado. Aún puede haber colisiones, aún puede suceder que dos experimentos queden acoplados por usar el mismo dado, pero la probabilidad ahora baja de 1 a 1/n (figura 33).

Figura 33. Evaluación de un experimento al que se ha asignado el dado 35. Se destaca la manera de obtener su valor.

```
<esi:choose>
  <esi:assign name="mi_dado" value="substr($(dado), 35*2, 2)"/>
  <esi:when test="$(mi_dado) < 50">
    <esi:assign name="experimento_1" value="TRATAMIENTO"/>
  </esi:when>
  <esi:otherwise>
    <esi:assign name="experimento_1" value="CONTROL"/>
  </esi:otherwise>
</esi:choose>
```

La segunda variación es la asignación, también a cada experimento, de un desplazamiento a aplicar en la lectura del dado. Así, en caso de una colisión, hay una instancia más de protección (figura 34).

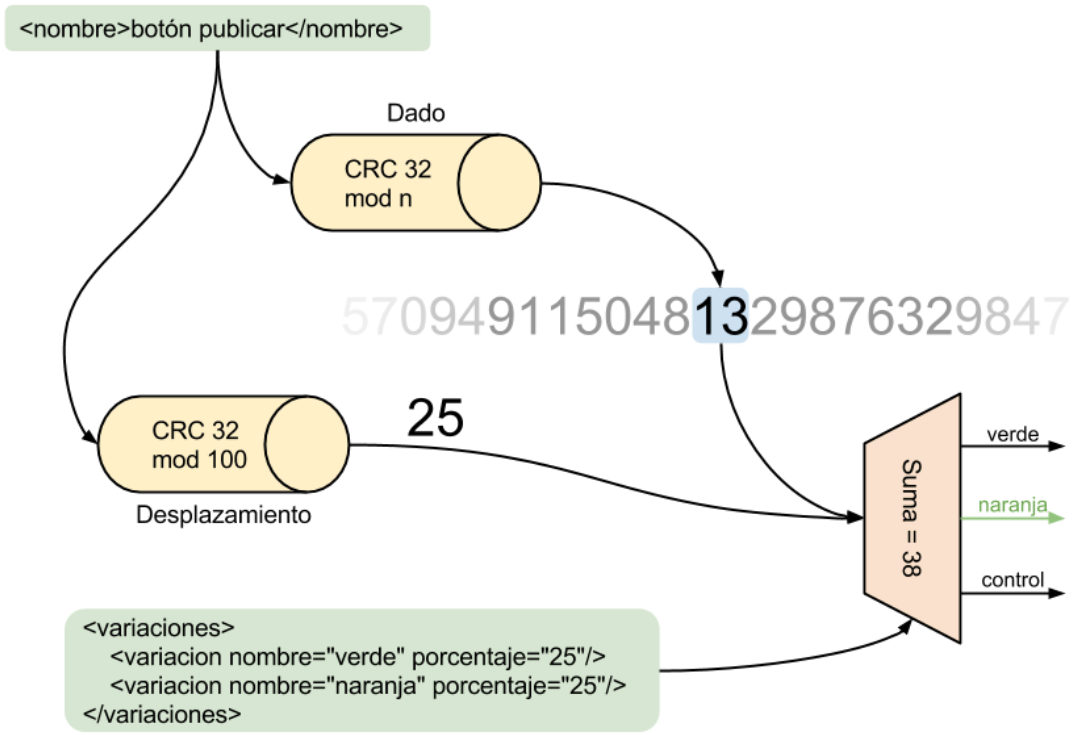
Figura 34. Evaluación de un experimento al que se ha asignado el dado 35 y el desplazamiento 68. Se destaca la manera de obtener el valor final.

```
<esi:choose>
  <esi:assign name="mi_dado" value="(substr($(dado), 35*2, 2) + 68) mod
100"/>
  <esi:when test="$(mi_dado) < 50">
    <esi:assign name="experimento_1" value="TRATAMIENTO"/>
  </esi:when>
  <esi:otherwise>
    <esi:assign name="experimento_1" value="CONTROL"/>
  </esi:otherwise>
</esi:choose>
```

La asignación de los dados y los desplazamientos debía ser uniforme, para minimizar la probabilidad de colisión de muestras entre experimentos, e invariable, para que pudiera ser calculada en función de un dato conocido y no fuese necesario almacenarla. Por su disponibilidad y bajo costo de ejecución, se usó el cálculo de redundancia cíclica de 32 bits (CRC32) como función de dispersión, y se aplicó sobre el nombre del experimento, que suele ser único y no variar durante la vida del mismo (figura 35).

Figura 35. Una función de dispersión distribuye uniformemente los experimentos entre los dados y en el espacio de valores de cada dado. Una vez determinados el dado y el desplazamiento, el valor resultante indica la variación para el usuario.

```
dispersión(experimento)     = crc32(experimento.nombre)
dado(experimento)           = dispersión(experimento) mod cantidadDados
desplazamiento(experimento) = dispersión(experimento) mod 100
```



El desplazamiento está restringido por definición a valores entre 0 y 99, pero la cantidad de dados está limitada sólo por el máximo espacio de almacenamiento en una cookie: 4096 bytes. Cuantos más dados se usen, menor será la probabilidad de colisión entre experimentos. Conviene además que esta cantidad comparta pocos factores primos con 100, para minimizar el acoplamiento entre las dos variaciones.

Este último enfoque funcionó correctamente y cubrió todos los requerimientos. Finalmente, actualizamos a una versión más reciente de Google Analytics, que ofrecía dimensiones personalizadas. Funcionalmente no ofrecían ninguna capacidad que no se tuviese ya, pero la interfaz de usuario para consultarlas era mucho más intuitiva y amigable, por lo que pasamos a segmentar por dimensiones.

3.3. Software desarrollado

3.3.1. Módulo de funciones de ESI

El código ESI es una mezcla de etiquetas XML e instrucciones en texto plano, que dificulta la lectura y la codificación del documento de salida. Además, como las instrucciones comparten el medio con sus propios argumentos, todas las cadenas de datos deben ser complementadas con caracteres de escape para que estos no interfieran con aquellas. Para evitar cualquier riesgo de confusión, se desarrolló una interfaz programática de infraestructura que representa las instrucciones de ESI, de manera que el desarrollador pueda desentenderse de las cuestiones propias de ese lenguaje y concentrarse en resolver su problema particular. Este módulo de funciones hace las veces de compilador transformando las instrucciones del lenguaje de programación, en este caso PHP, a las instrucciones de la infraestructura, en este caso ESI, automáticamente.

Si bien este módulo ayuda a la resolución del problema, no es estrictamente parte de él y por eso es considerado infraestructura. En una pieza de software más grande, puede ser perfectamente utilizado por otros módulos con otros objetivos.

3.3.2. Módulo de funciones de variación de contenido

Este es el desarrollo central y la contribución principal de este trabajo. Consiste en un módulo de funciones que abstrae la creación de un documento de salida de texto plano (usualmente HTML, pero funcionará igualmente para Javascript o CSS) con fragmentos condicionales, cuya presencia o ausencia en el documento que recibe el cliente será determinada en el futuro, a cargo de la red de entrega de contenidos.

Puesto que el contenido de salida de la aplicación que use el módulo debe ser envuelto en código ESI para la CDN, el módulo utiliza el patrón de inversión de control, con el cual el código principal a ejecutar es el módulo mismo y, como parámetro, se le entrega una

referencia a función con el código interno de la aplicación. Dentro de esta función, está expuesta la interfaz del módulo con las primitivas necesarias para condicionar los bloques.

La definición de los experimentos es declarativa, de manera que toda la lógica queda abstraída dentro del módulo y el desarrollador no necesita conocerla. Esta forma también permite tomar esta información declarativa de una fuente externa, como una base de datos o un archivo de configuración, para el que se puede crear una herramienta administrativa que incluso un usuario ajeno al departamento de tecnología pueda operar.

La rutina de tracking está separada en un submódulo propio, de manera que pueda ser reemplazada por otros métodos en un futuro. El método elegido para la presente implementación es la integración con Google Analytics, que es la pieza complementaria del variador, donde el operador puede ver los resultados de los experimentos.

3.3.3. Prueba de concepto de variación en el cliente

Es un documento HTML con fragmentos de código Javascript incluido y fragmentos separados para incluir en tiempo de carga, en el cual el orden de los elementos, un orden particular pero no alejado de la forma habitual de uso del guionado, pone en evidencia la condición de carrera entre el hilo de construcción del modelo de objetos del documento HTML y el hilo de presentación en pantalla, haciendo que se vea durante tiempo suficiente la versión original del documento antes de ser modificada por una rutina de modificación para experimentos. La demora entre los eventos está exagerada artificialmente para que pueda ser vista claramente por el usuario, pero sólo con una instrucción `sleep`. En una situación real, la misma demora puede provenir de cualquier elemento de la red que une el servidor con el cliente.

3.3.4. Sitio de pruebas con cliente automático

El sitio de pruebas es una aplicación web que genera un simple sitio web de avisos clasificados con una circulación equivalente a la de OLX. Es el autómata finito de cuatro estados de la figura 26: listado de avisos, detalle de un aviso, formulario de publicación y formulario de contacto al vendedor.

El cliente automático es un guión Javascript que usa la herramienta de navegación CasperJS para pedir las páginas del sitio de pruebas y circular entre ellas. Para guiar la circulación, cada vínculo entre páginas tiene un peso asociado que indica la probabilidad de

seguirlo. Cada corrida del cliente automático simula una *visita* de un usuario, en la que se ven una o más páginas.

El sitio de pruebas tiene un experimento A/B en su página inicial: el listado de avisos. Cada visita del cliente automático se comportará como un nuevo usuario, para el cual se elegirá aleatoriamente la versión de tratamiento o de control. Los resultados de las corridas pueden verse inmediatamente en una cuenta de pruebas de Google Analytics.

4. Resultados

Como resultado del trabajo desarrollado, se construyó una herramienta que permite aplicar la técnica de experimentación A/B para optimizar el rendimiento de un sitio web generado por una aplicación escrita en lenguaje PHP. Cumple los requisitos mínimos de cualquier herramienta que tenga ese objetivo, como poder definir los experimentos, exponerlos a fracciones de la población, medir variables de negocio en cada variación y reportar los resultados, pero ciñéndose en su implementación a los límites impuestos por una arquitectura más compleja que aquella para la cual están desarrolladas las herramientas más populares.

El escenario en el que se enmarcó el desarrollo es una arquitectura cliente-servidor pero con un intermediario transparente que representa al cliente ante el servidor y al servidor ante el cliente, constituyendo de hecho una arquitectura de tres capas donde las dos interfaces entre capas son iguales. La solución consistió en darle un nuevo rol al intermediario, convirtiéndolo en un agente activo y delegando en él la tarea principal: la variación del contenido que forma parte de un experimento A/B.

El servidor de origen puede generar documentos de salida cuyo contenido sea constante, que no varíe en función de quién lo ha pedido. De esta manera, los documentos son reutilizables y cada servidor de la red de entrega de contenidos puede guardar copias y no pedirlos de nuevo, conservando en el servidor de origen las condiciones iniciales de mínimo trabajo. Este no ejecuta las instrucciones de variación de contenido, sino que sólo las declara para que sean ejecutadas más tarde, igual que cuando las variaciones ocurren en el cliente.

El cliente, a su vez, recibe el documento particular de la variación que le ha sido aleatoriamente asignada, listo para ser mostrado al usuario, sin contenido original que pueda ser visto por el usuario ni modificaciones pendientes por las que haya que ganar la condición de carrera. Este tampoco ejecuta las instrucciones de variación de contenido, sino que estas ya están ejecutadas en una instancia anterior, igual que cuando las variaciones ocurren en el servidor.

La interfaz programática de la herramienta resultó además no invasiva. El llamado a sus funciones sólo es requerido para envolver los bloques de código de la aplicación sujetos a variación, y sus argumentos son los mínimos: el experimento y la variación a la que

corresponde cada bloque. De esta manera, al finalizar un experimento, no hay nada que reimplementar: sólo hay que quitar los bloques de las variaciones perdedoras y el condicionamiento de la ganadora, tarea que es, además, fácilmente automatizable.

El conteo de las exposiciones y las conversiones con una herramienta externa permitió ahorrar tanto costo de desarrollo, al no implementar el receptor del conteo, como costo de producción, al no tener que soportar el voluminoso tráfico de tal recepción, y compatibilizar el reporte de los resultados con el resto de los reportes de web analytics en curso. No hubo grandes concesiones en el uso de esta herramienta externa, más que adaptarse a su interfaz programática y sus definiciones de funcionamiento interno. Para el objetivo del proyecto, no tendría sentido implementar de cero un receptor de conteo propio.

En el marco de la organización, la incorporación del guionado de la CDN se destaca como aporte. Siendo un medio para incorporar la CDN a la arquitectura, al conjunto de componentes cuyo comportamiento se puede definir para servir a los objetivos de tecnología, extiende los límites de acción del ingeniero de performance y abre la posibilidad a nuevas escalas de servicio. Hablando más específicamente de la herramienta, la distribución automática de los espacios de probabilidad elevó el nivel corriente de abstracción y permitió descargar tanto al equipo de desarrollo como al de marketing de la distribución manual, para la que se requieren ciertos conocimientos de matemáticas con los que no siempre se cuenta.

La implementación clásica de experimentación A/B de tirar un nuevo dado para cada experimento y utilizar su valor literal fue redefinida por una nueva implementación de tirar un solo dado para todos los experimentos e interpretarlo de una manera nueva en cada uno. Esto permitió que del lado del cliente, el componente de arquitectura sobre el que menos control se tiene, se pudiera mantener una pieza de información constante y de validez ilimitada, evitando tener que correr rutinas de limpieza o refresco, que introducen nuevas dificultades y cuya completitud nunca se puede garantizar.

Cabe destacar que el traslado del comportamiento al frente, si bien reduce la carga en los servidores de origen, la aumenta en la CDN. La suma de trabajo sigue siendo menor, pues la tasa de reutilización de objetos y fragmentos de objetos aumenta, pero aun así la CDN podría no estar preparada tal aumento, o su costo monetario podría superar el costo de seguir soportando de manera subóptima la carga en el origen. Usualmente, las CDNs están preparadas para un procesamiento a una escala mucho mayor que las de los orígenes y el impacto es mínimo. Además, su servicio suele estar garantizado por un acuerdo de nivel de

servicio (o *Service Level Agreement* o *SLA*) [26] y entonces el impacto está ya contemplado previamente.

5. Conclusión y trabajo futuro

En el escenario de un sitio web cuyo contenido es distribuido globalmente por una red de entrega, las opciones tradicionales de implementación de la técnica de experimentación A/B no permiten conservar la capacidad de servicio o la calidad del producto, por lo que se requiere una adaptación técnica. El presente trabajo describe el estado del arte en implementaciones en servidor y en cliente y propone una implementación alternativa utilizando nuevas capacidades de las redes de entrega de contenido.

Las redes de entrega de contenido nacieron como, y han sido tradicionalmente, redes de servidores de solamente *caching*, activos en términos de transferencia y almacenamiento, pero pasivos en términos de procesamiento de información. De allí su nombre: se dedican la entrega, a la última tarea de producción, al trabajo de la última milla. Es así que habitualmente no son consideradas parte de la arquitectura de un servicio web, sino un servicio de infraestructura, como la alimentación eléctrica de los servidores o el acceso a la misma Internet.

La contribución más inmediata y palpable de este trabajo es la resolución técnica de la necesidad en el escenario propuesto, que resultó satisfactoria para los objetivos planteados, aun dejando cuestiones abiertas a discusión y mejora. Tiene además la casual fortuna de alcanzar el mundo real en el marco de una organización comercial que ya le está dando utilidad.

La contribución menos evidente, pero quizás más importante, es el inicio de un camino, que pocos están transitando aún, hacia la activación de la red de entrega de contenido. Guionar la red de entrega desde el origen y darle comportamiento es incorporarla a la arquitectura. La arquitectura pasa de medirse en decenas y a medirse en miles, pasa de ser un grupo de actores conocidos a ser una muchedumbre comandada a control remoto, al servicio de la aplicación central, y eleva ya no sus cantidades sino su orden de magnitud, invitando al arquitecto y al ingeniero a un nuevo mundo de problemas y soluciones.

5.1. Trabajo futuro

Ya prevista en la organización del código está la posibilidad de soportar otros medios de registro de las acciones del usuario. La opción elegida para el desarrollo es el registro del lado del cliente usando Google Analytics. Hay otras opciones del lado del cliente, otras del

lado del servidor, y hasta se podría desarrollar una nueva en la CDN. Si bien el registro es necesario, cualquier opción es válida y otras opciones podrían ser evaluadas, tanto para mejorar el tiempo de *rendering* ante el usuario como para reducir el impacto en la arquitectura de tan voluminoso tráfico. Para cualquiera que use una CDN con la intención de descargar el origen, es probable que el registro en el servidor no sea una opción, pero no por ello la interfaz tiene que estar limitada.

Otros métodos para la distribución de probabilidades también pueden ser vías de mejora. El método elegido para este trabajo es el más básico: una distribución declarativa estática. Existen métodos que distribuyen las probabilidades dinámicamente, como el Multi-armed Bandit [27], que ajusta constantemente las probabilidades en función de los resultados parciales, dando más tráfico a las variaciones ganadoras para aumentar la ganancia total en el tiempo, sin inhibir las restantes mientras los resultados no sean determinantes. Hay incluso distintos algoritmos para la distribución dinámica [28]. Estos métodos son mejores para el negocio que los utiliza, pues aumentan la ganancia durante el período de prueba, pero son también más complejos de implementar y entonces el costo a amortizar es mayor.

Finalmente, queda abierta la puerta a nuevas ideas de delegación de comportamiento a la CDN. Como se mencionaba más arriba, el registro de las acciones del usuario es una. El campo ha sido fértil hasta ahora y está sin explorar.

Bibliografía

[1] Mendenhall, W.; Beaver, R.; Beaver, B. (2012). "Introduction to probability and statistics". Cengage Learning.

[2] Ioannidis, J. P. A. (2005). "Why Most Published Research Findings Are False". PLoS Med 2(8): e124.

[3] Johnson, R. A.,; Wichern, D. W. (2002). "Applied multivariate statistical analysis". Upper Saddle River, NJ: Prentice hall, 5(8).

[4] Cox, D. R. (1958). "Planning of experiments". Oxford, England: Wiley.

[5] Nygren, E.; Sitaraman, R. K.; Sun, J. (2010). "The Akamai Network: A Platform for High-Performance Internet Applications". ACM SIGOPS Operating Systems Review 44(3): pp. 2-19.

[6] Tsimelzon, M.; Weihl, B.; Chung, J.; Frantz, D.; Brasso, J.; Newton, C.; Hale, M.; Jacobs, L.; O'Connell, C. (2001). "ESI Language Specification 1.0". W3C. http://www.w3.org/TR/esi-lang. Retrieved 2014-02-12.

[7] Grosskurth, A.; Godfrey, M. W. (2005). "A reference architecture for Web browsers". Proceedings of the 21st IEEE International Conference on Software Maintenance, 26-29 Sept. 2005, pp. 661-664.

[8] Custom Dimensions & Metrics - Feature Reference - Google Analytics - Google Developers. [https://developers.google.com/analytics/devguides/platform/customdimsmets].

[9] Best Practices for Speeding Up Your Web Site - Yahoo Developer Network. [https://developer.yahoo.com/performance/rules.html].

[10] Problema de la parada - Wikipedia. [http://es.wikipedia.org/wiki/Problema_de_la_parada].

[11] How Loading Time Affects Your Bottom Line - KissMetrics. [https://blog.kissmetrics.com/loading-time/].

[12] Inversion of control - Wikipedia. [http://en.wikipedia.org/wiki/Inversion_of_control].

[13] Anuncios gratis en Argentina - OLX. [http://www.olx.com.ar].

[14] Tiempo de acceso - Wikipedia. [http://es.wikipedia.org/wiki/Tiempo_de_acceso].

[15] HTTP Compression - Wikipedia. [http://en.wikipedia.org/wiki/HTTP_compression].

[16] Time To First Byte - Wikipedia. [http://en.wikipedia.org/wiki/Time_To_First_Byte].

[17] Red de entrega de contenidos - Wikipedia.
[http://es.wikipedia.org/wiki/Red_de_entrega_de_contenidos].

[18] Proveedor de Servicios de Cloud Computing y Red de Entrega de Contenidos - Akamai.
[http://www.akamai.com/].

[19] JavaScript - Wikipedia. [http://es.wikipedia.org/wiki/JavaScript].

[20] Web Analytics - Wikipedia. [http://en.wikipedia.org/wiki/Web_analytics].

[21] Sitio web oficial de Google Analytics: Analítica web e informes - Google Analytics.
[http://www.google.com/analytics/].

[22] Caché web - Wikipedia. [http://es.wikipedia.org/wiki/Cach%C3%A9_web].

[23] AWS. Amazon CloudFront CDN - Content Delivery Network & Streaming.
[http://aws.amazon.com/cloudfront/].

[24] Server Side Includes - Wikipedia. [http://es.wikipedia.org/wiki/Server_Side_Includes].

[25] Akamai ESI Extensions - Akamai Documentation.
[http://www.akamai.com/dl/technical_publications/akamai_esi_extensions.pdf].

[26] Acuerdo de nivel de servicio - Wikipedia.
[http://es.wikipedia.org/wiki/Acuerdo_de_nivel_de_servicio].

[27] Multi-armed bandit - Wikipedia. [http://en.wikipedia.org/wiki/Multi-armed_bandit].

[28] Volodymyr Kuleshov and Doina Precup. Algorithms for the multi-armed bandit problem.
[http://www.cs.mcgill.ca/~vkules/bandits.pdf].